

Tutorial: Generating Simulated Traces of Hardware AES for Pre-Silicon Side-Channel Analysis (Version 2)

Phuc Mai, Logan Reichling, Muhib Khan, and Boyang Wang

Data and System Security Lab (DaSec), ECE Department, University of Cincinnati

Last Modified: September 2026, Email: boyang.wang@uc.edu

What is New in Version 2. This is the second version of this tutorial. Compared to our first version, which we used SecWork (unmasked AES) and BRAM (masked AES), we replace them with ProjectVault (unmasked AES) and SMAesH (masked AES) as the two target implementations. We are also able to implement each of the new target on a ChipWhisperer CW305 FPGA board to validate/align results from post-silicon side-channel analysis in Version 2 (we were not able to achieve with the two targets in version 1). Details regarding how to implement each new target design on CW305 board are included in this Version 2

Associated Publications. Some of our research findings based on the datasets collected from this tutorial are published in the following paper: *Logan Reichling, Phuc Mai, Muhib Khan, Kaiden Thomas, Boyang Wang, "HardSide: Pre-Silicon Side-Channel Analysis for Hardware AES with Deep Learning," IEEE International Conference on Physical Assurance and Inspection of Electronics (PAINE 2026), Phoenix AZ, October 27-29, 2026.*

1 Introduction

Overview. This document explains how to generate simulated traces, especially at the RTL (Register Transistor Level), for hardware implementations of AES (Advanced Encryption Standard) that target FPGAs. The simulated traces obtain from one hardware implementation can be utilized for *pre-silicon side-channel analysis* to examine *whether* this hardware AES implementation carries side-channel leakage at the design stage of hardware and also *in which part* of the design the leakage happens. If side-channel leakage exists, then it is *theoretically* feasible for an attacker to recover encryption keys from the chips fabricated based on this design, even though encryption algorithms are mathematically secure. Being able to generate these traces and perform pre-silicon side-channel analysis can help designers verify/find security vulnerabilities, mitigate them, and enhance hardware security of chips at the design stage. It is worth mentioning that this is a *white-box analysis*, which means we need to have access to the source code of the hardware design.

This tutorial (current version) investigates multiple hardware AES implementations, including *one unmasked AES* and *one masked AES*, and describes the simulation pipeline for each hardware implementation.

Why This Document is Needed? While there are tools (both commercial and academic tools) available to generate simulated traces of hardware designs at the RTL in general, there

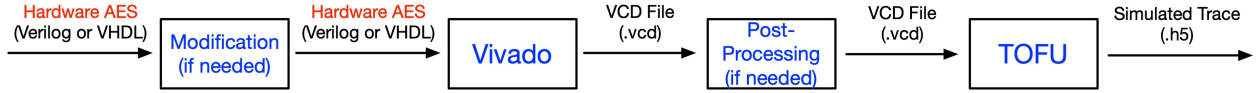


Fig. 1: Overview of our simulation pipeline

are no public or well-illustrated documents, specifically for AES hardware implementations, that researchers and students can leverage to easily perform pre-silicon side-channel analysis. We believe that the lack of such documents is due to the complicated tool chains for hardware designs and the challenges in the integration of simulation and encryption. Specifically, the integration of the simulation process of an encryption design requires comprehensive knowledge and strong hands-on skills in (at least) three critical areas, including *hardware design*, *cryptography algorithms*, and *side-channel analysis*. It can often be challenging to researchers and students who are new to this area¹. This document aims to fill this gap in education, enhance research reproducibility, and broaden the participation in pre-silicon side-channel analysis.

Summary of Our Simulation Pipeline. In order to generate simulated traces for analysis, we leverage the following workflow: An AES hardware implementation (written in Verilog or VHDL) is selected and then loaded into Xilinx’s Vivado design software [2]. Given a plaintext and a key, a VCD (Value Change Dump) file is generated from the Vivado simulation of one execution of the AES hardware implementation at the RTL (i.e., Vivado behavioral simulation)².

Given the VCD file, an open-source tool named TOFU [8] is leveraged to generate a simulated trace. TOFU produces a simulated trace based on toggle counts, i.e. the total number of signal changes (0s $\rightarrow$ 1s and 1s $\rightarrow$ 0s) in the hardware design execution, at each simulation timestamp listed in the provided VCD file. TOFU provides both Hamming Weight model or Hamming Distance model when generating a simulated trace. Specifically, Hamming Weight estimates the power consumption at timestamp i based on the number of signals that are at 1 (i.e., high) at timestamp i (see line 100 at *synthesize.py* in TOFU repository). Hamming distance estimates the power consumption at timestamp i based on the number of signals that are toggled/switched between timestamp i and timestamp $i - 1$. When multiple simulated traces is needed, each trace is produced from one VCD file by repeating the above steps but changing the plaintext and/or key in the testbench file.

Special Note 1. In our example, *multiple lines in the original implementation of an AES hardware implementation (e.g., ProjectVault) needs to be **carefully modified by us** and an **additional customized post-processing on VCD files** is required in order to integrate it into the entire simulation pipeline and being able to collect simulated traces in a large scale automatically for downstream side-channel analysis.*

¹ At least for us, it took several months to investigate, establish, and polish the simulation pipeline, given the previous experience we already had on cryptography algorithms and side-channel analysis.

² Depending on the implementation, we could further generate a VCD file, which includes the results of the simulation at the netlist level (i.e., Vivado post-synthesis simulation)

Special Note 2. There are different ways to generate simulated traces for AES at the RTL level. We present one way that works for us in this document and we acknowledge that this may not be the most effective way to achieve the goal.

Source Code and Example Datasets. In addition to the description mentioned in this document, we also provide source code, tutorial videos, and links to example datasets on our GitHub repository [1](<https://github.com/UCdasec/HardSide>) for reproducibility. We recommend that readers read this tutorial document first, and then review the videos. We have another research paper [7] describing the detailed results of pre-silicon side-channel analysis over the two example datasets using multiple methods, including TVLA (Test Vector Leakage Assessment), CPA (Correlation Power Analysis), Template Attacks, and Deep Learning Side-Channel Analysis.

Acknowledgments. The research and education efforts associated with this document are supported by CHEST – NSF IUCRC Center in Hardware and Embedded Systems Security and Trust. The education effort is also partially supported by Ohio Cyber Range Institution.

2 Software and Hardware Settings

We use the following hardware and software in our simulation pipeline.

- Hardware: A desktop with an Intel i7 CPU, 24 GB memory, and an Intel HD Graphics 630 GPU running Linux (Ubuntu 22.04)
- Software (all free or open-source): Xilinx Vivado 2023.2 ML Edition, Python 3, h5py library, ProjectVault [6], SMAesH [3], TOFU [8], GCC, libtinfo5

3 Software Installation

We briefly describe how to install python, Vivado, and TOFU below. In addition, we also discuss where/how to download ProjectVault, which is an open-source unmasked AES written in Verilog.

3.1 Install Python 3 and Dependencies

We install python3 using the command below:

```
1 sudo apt install python3
```

Install the necessary dependencies as below:

```
1 apt-get install python3-pip
2 pip install h5py matplotlib numpy
3 sudo apt install libtinfo5
```

3.2 Install TOFU

Download and extract TOFU from <https://gitlab.lrz.de/tueisec/tofu>. Once the download is completed, extract the ZIP file to our home directory. This creates a directory named `./tofu-master`. The entire TOFU directory consists of multiple folders and files, as shown in Fig. 3. TOFU takes a VCD file as input and outputs a simulated trace. We will present details regarding how to generate a trace in our case later.

```
muhibkhan@ucsec-OptiPlex-7050:~/Desktop/tofu-master$ ls
engine      example-kat  LICENSE      pyproject.toml  setup_attack_framework.sh  synthesize.py
example     extractsignalids.py  Makefile     README.md       setup_lascar.sh            tueisec.py
example-aes-cortex  helper.py      ntofu.py     selftest.py     setup_tofu_python.sh      value.py
example-aes-vhdl   HOWTO.md      parse.py     selftest.sh     signals.py
```

Fig. 3: Files contained in TOFU’s directory (`./tofu-master`)

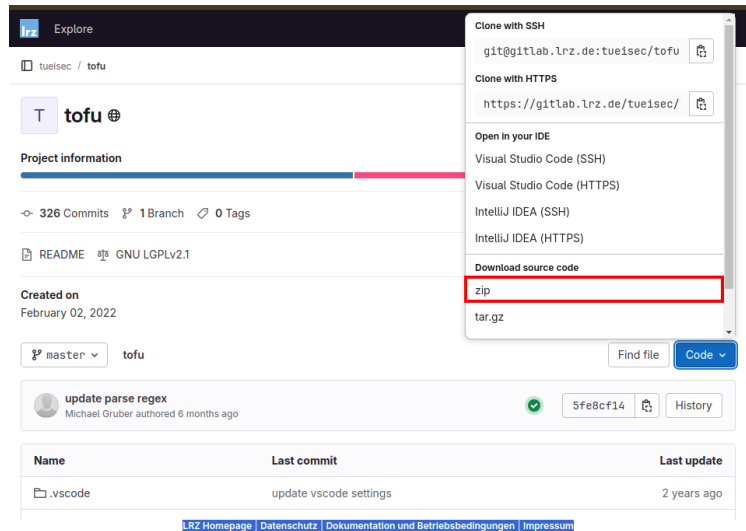


Fig. 2: Download TOFU

3.3 (Optional) Install GTKWave and use it to view the VCD file

This step is optional. We can install GTKWave and view the VCD file later.

1. Install GTKWave

```
1 sudo apt install gtkwave
```

3.4 Download and Install Vivado 2023.2 ML Edition

(1) Download AMD unified installer for Vivado ML Edition 2023.2 (Full Product Installation) from the Xilinx website ³.



Fig. 4: Vivado Installer Download

³ <https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/vivado-design-tools/2023-2.html>

(2) Once click the download link, we will need to log in or create an AMD/Xilinx account (if we do not have) and provide information, such as name, address, phone number, etc. Then scroll down to the bottom of the page to download the bin file as shown in Fig. 5

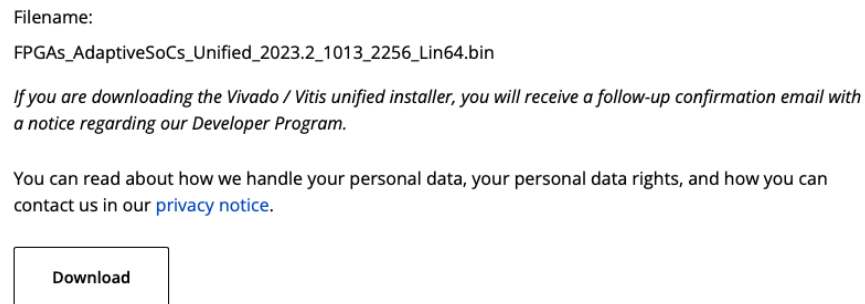


Fig. 5: Download the bin file of Vivado Installer

(3) Once the download is completed, to execute the bin file run `sudo chmod +x` command in the terminal to make the bin file executable, and then execute it.

```
1 sudo chmod +x FPGAs_AdaptiveSoCs_Unified_2023.2_1013_2256_Lin64.bin
2 sudo ./FPGAs_AdaptiveSoCs_Unified_2023.2_1013_2256_Lin64.bin
```

```
test_pico@ucsec-OptiPlex-7050:~/Downloads$ sudo chmod +x FPGAs_AdaptiveSoCs_Unified_2023.2_1013_2256_Lin64.bin
test_pico@ucsec-OptiPlex-7050:~/Downloads$ ./FPGAs_AdaptiveSoCs_Unified_2023.2_1013_2256_Lin64.bin
Verifying archive integrity... All good.
Uncompressing AMD Installer for FPGAs and Adaptive SoCs.....
.....
.....
.....
.....
.....
.....
.....
.....
This is a fresh install.
INFO Could not detect the display scale (hdpi).
    If you are using a high resolution monitor, you can set the insaller scale factor like this:
        export XINSTALLER_SCALE=2
        setenv XINSTALLER_SCALE 2
INFO - Started in: 2 Sec
INFO - Internet connection validated, can connect to internet.
```

Fig.6: Vivado Install Commands

(4) The installer should pop up, click next and then continue to login by providing email address and password, and click Next

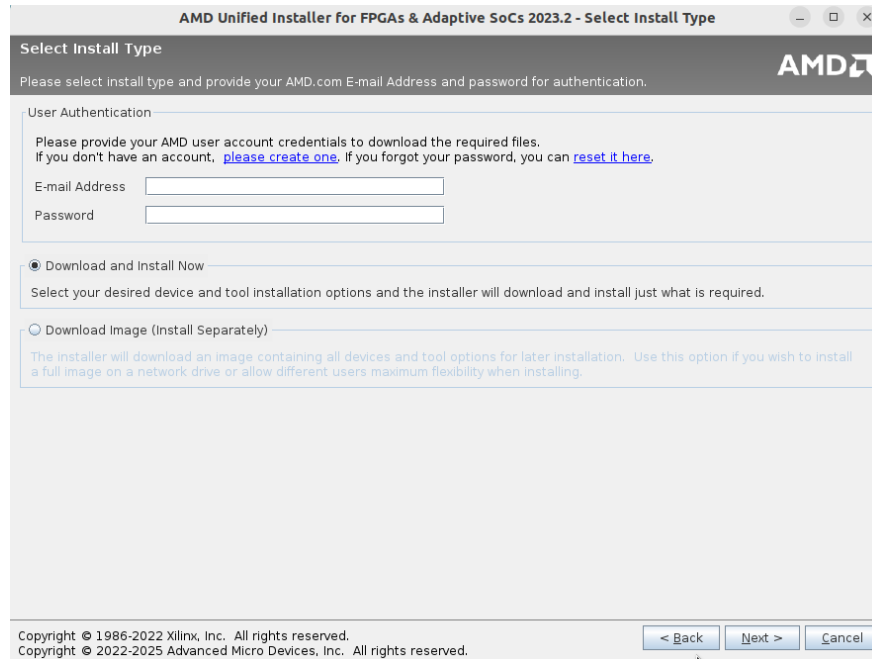


Fig. 7: Installer login page

(5) Select Vivado and click Next (we only need Vivado for examples in this document)

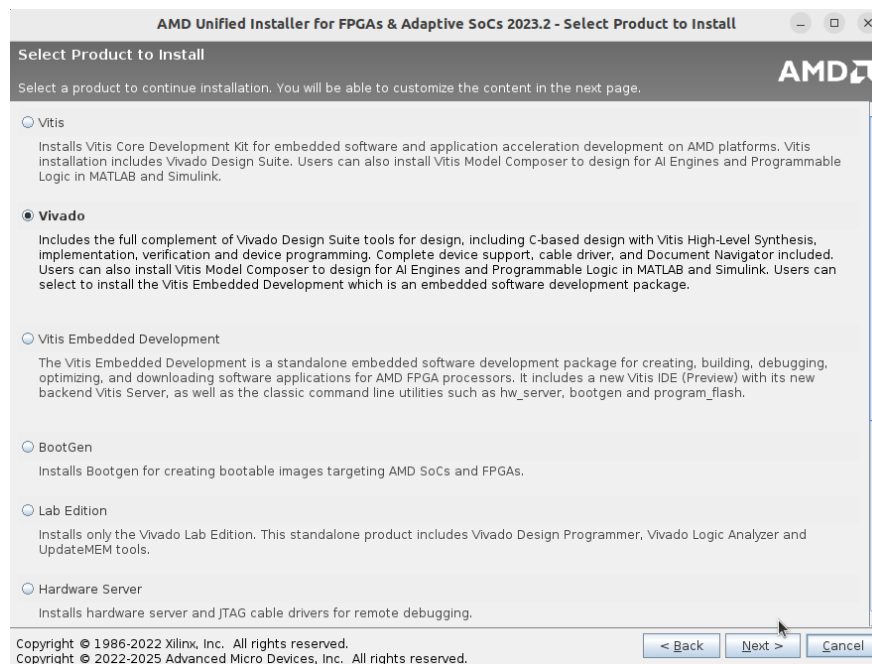


Fig. 8: Select Vivado

(6) Select Vivado ML Standard Edition and click Next

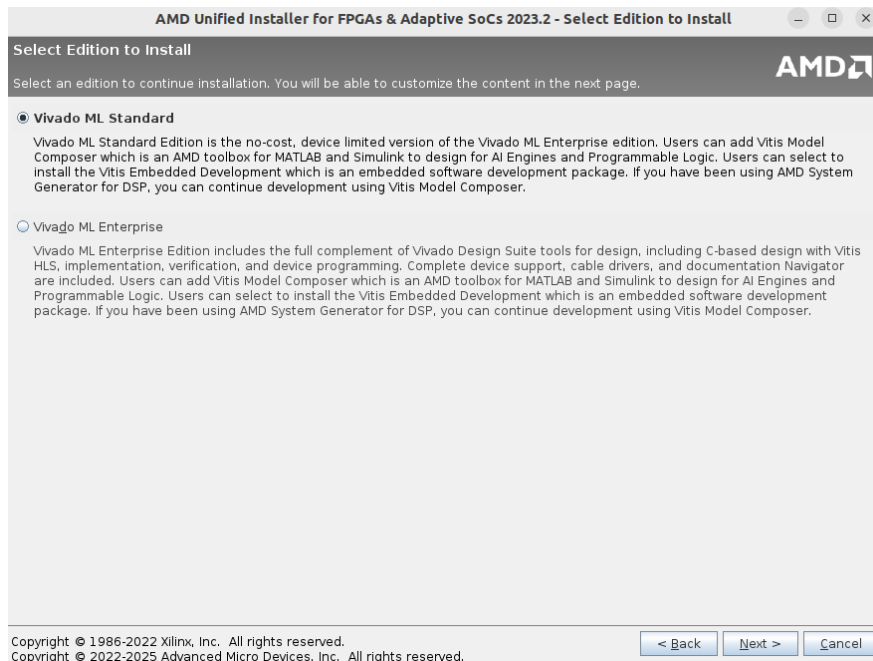


Fig. 9: Vivado Edition

(7) Select the options shown in the following screenshot and click Next;

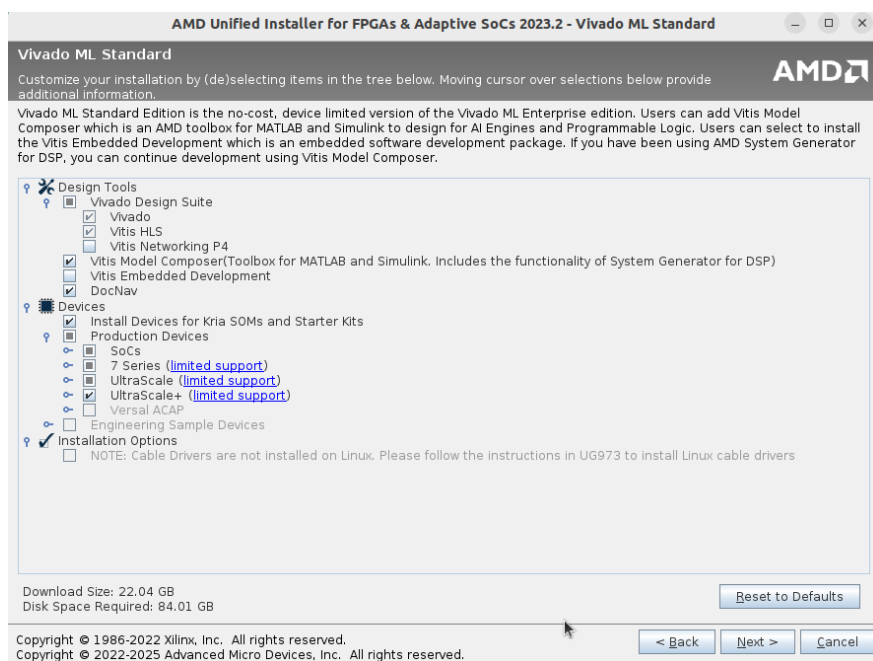
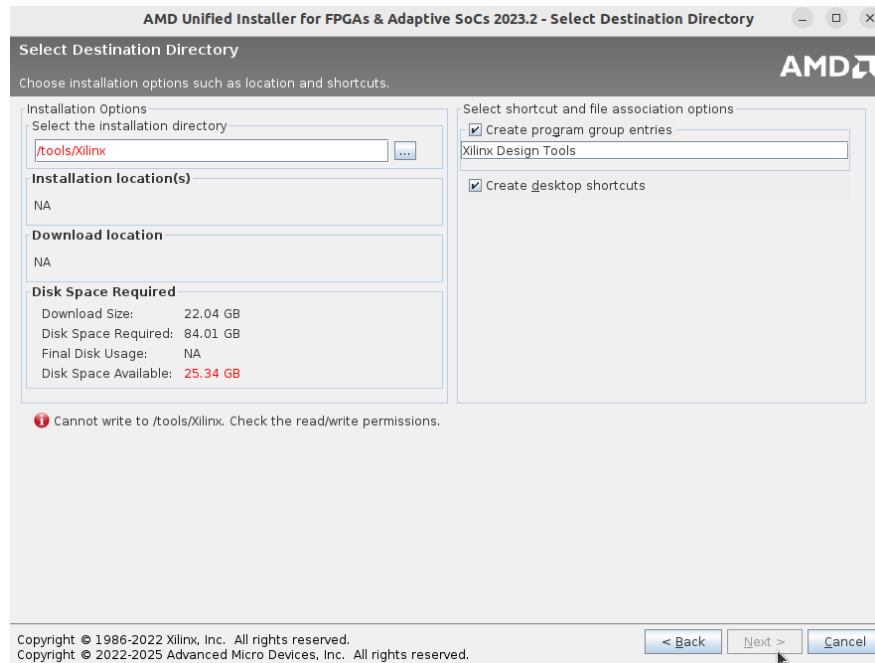


Fig. 10: Vivado Install Options

(10) Next, select the install directory and click Install. The installation may take 20~30 mins to complete.



3.5 Download the Source Code of ProjectVault

(1) Download the ZIP file from the GitHub repository of ProjectVault AES at <https://github.com/ProjectVault/orp>.

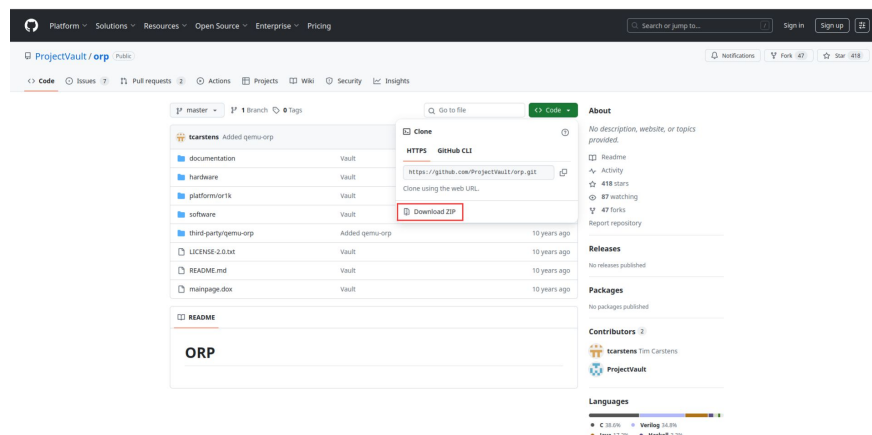


Fig. 11: Download the source code of ProjectVault.

(2) Extract the ZIP file to your home directory. This creates a directory named `./orp-master`. The entire design of ProjectVault consists of multiple folders (Fig. 12). The source code of the main design can be found by following the path `./orp-master/hardware/mse1SoC/src/systems/geophyte/rtl/verilog/crypto_aes/rtl/verilog/` (Fig. 14). The testbench files can be found by following the path `./orp-master/hardware/mse1SoC/src/systems/geophyte/rtl/verilog/crypto_aes/bench/verilog/` as shown in Fig. 13.

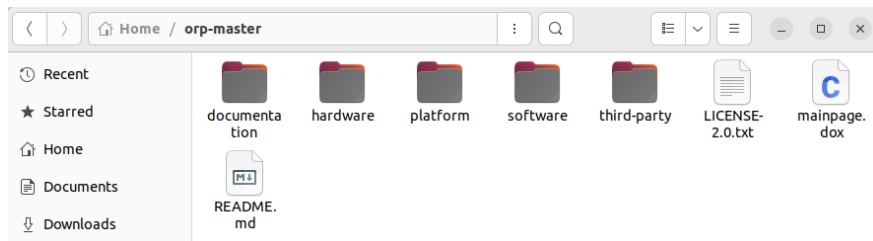


Fig. 12: Extract the zip file of ProjectVault

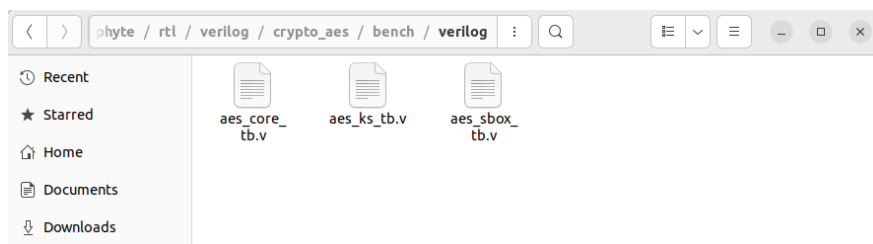


Fig. 13: Testbench files in ProjectVault

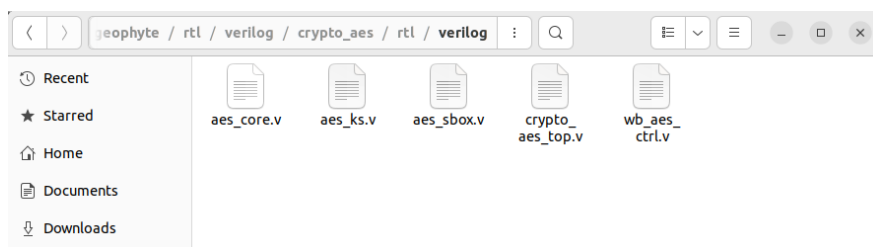


Fig. 14: Source files in ProjectVault

4 Generating Simulated Traces for Unmasked AES

In this section, we describe the detailed steps for establishing our simulation pipeline and how to generate simulated traces at the RTL level by passing a key and plaintexts to the unmasked AES hardware implementation we use (i.e., ProjectVault).

Overview. We will first create a Vivado project based on the source code of ProjectVault. Once the project is created, we will need to modify the original code properly to make it compatible with downstream parsing tool (TOFU), and run RTL simulation to obtain a VCD file. Once a VCD file is obtained, we will install and run TOFU to generate a simulated trace. We describe the details of each step below.

4.1 Create a Project in Vivado for AES Implementation

Once we have Vivado installed, we need to create a new project for the AES hardware implementation (ProjectVault) by following the steps below.

1. Launch Vivado 2023.2 and click on “Create Project”.
2. Follow the “Create New Project” wizard, create a project name (`ProjectVault_AES_Impl` in our example), set the location to home directory: `/home/username` (username is `mdphuc` in this running example), check the box `Create project subdirectory`, and click “Next”

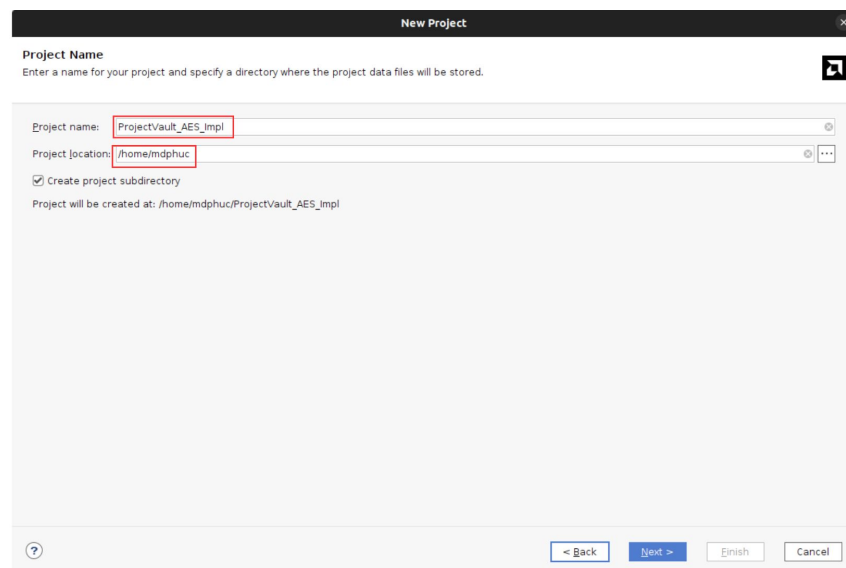


Fig. 15: Step 2: Create project name and choose project location

3. Specify the project type as an RTL Project, and click ”Next”

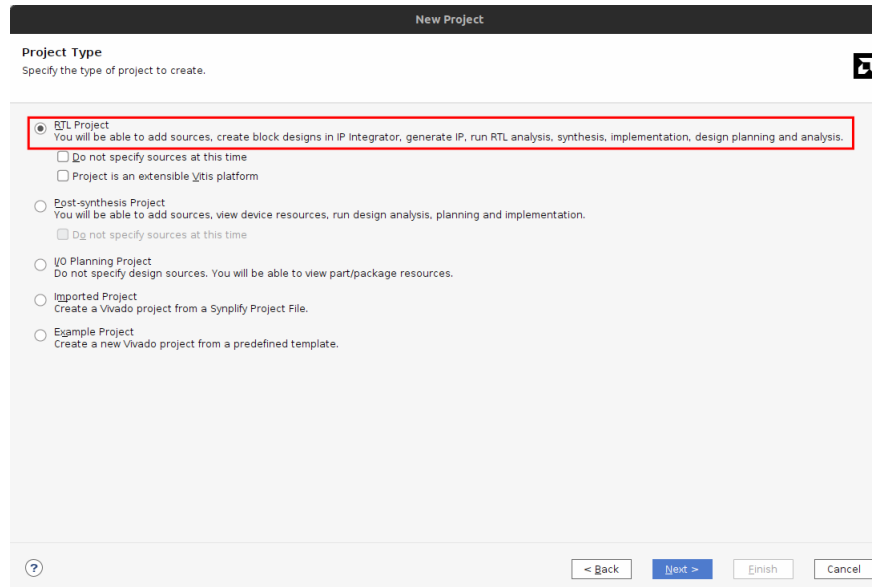


Fig. 16: Step 3: Choose project type as an RTL project

- On the “Add Sources” screen, click “Add Files” and add the 3 source files, including `aes-core.v`, `aes_ks.v`, `aes_sbox.v`, from `src/rtl` within the `aes-master` directory to the project, then click “OK” to return to the “Add Sources” screen.

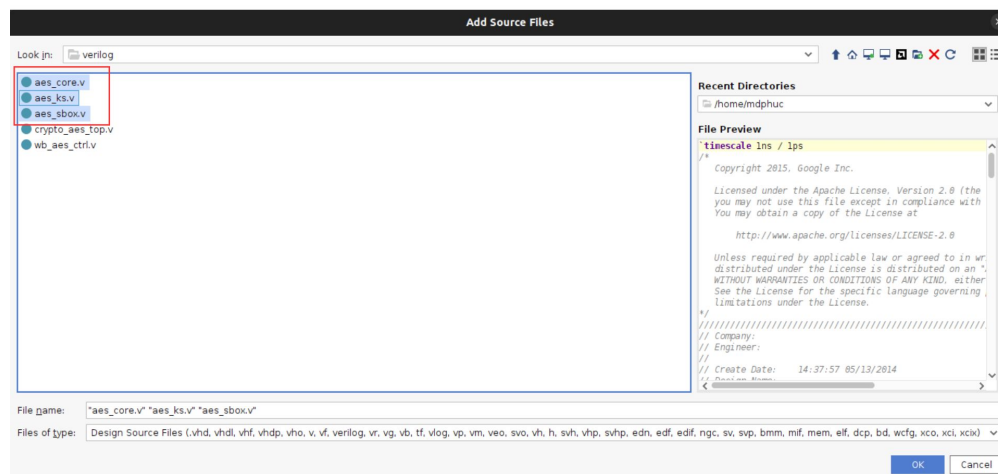


Fig.17: Step 4: Add 3 source files of AES from ProjectVault to the project

5. Press “Add Files” again to add the testbench file `tb_aes.v` only from `src/tb/` within the `aes-master` directory to the project, then click “OK” to return to the “Add Sources” screen.

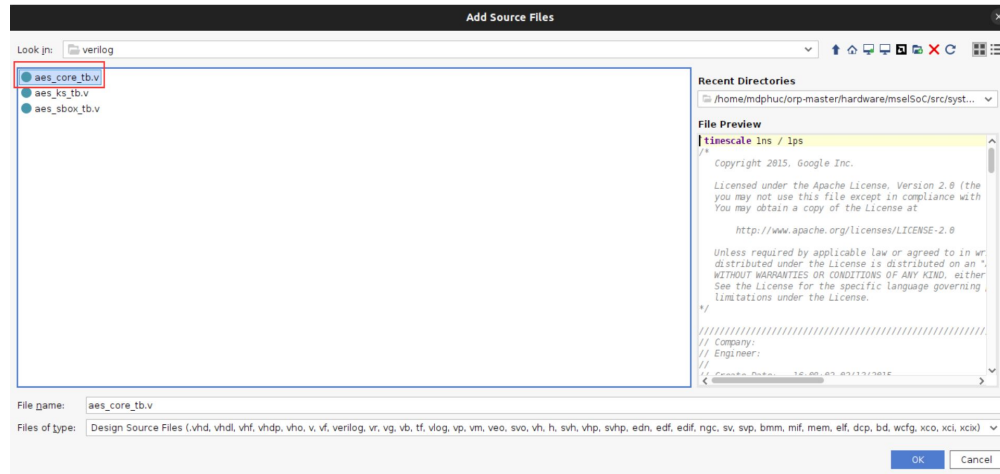


Fig. 18: Step 5: Add the testbench file `aes_core_tb.v` from ProjectVault to the project

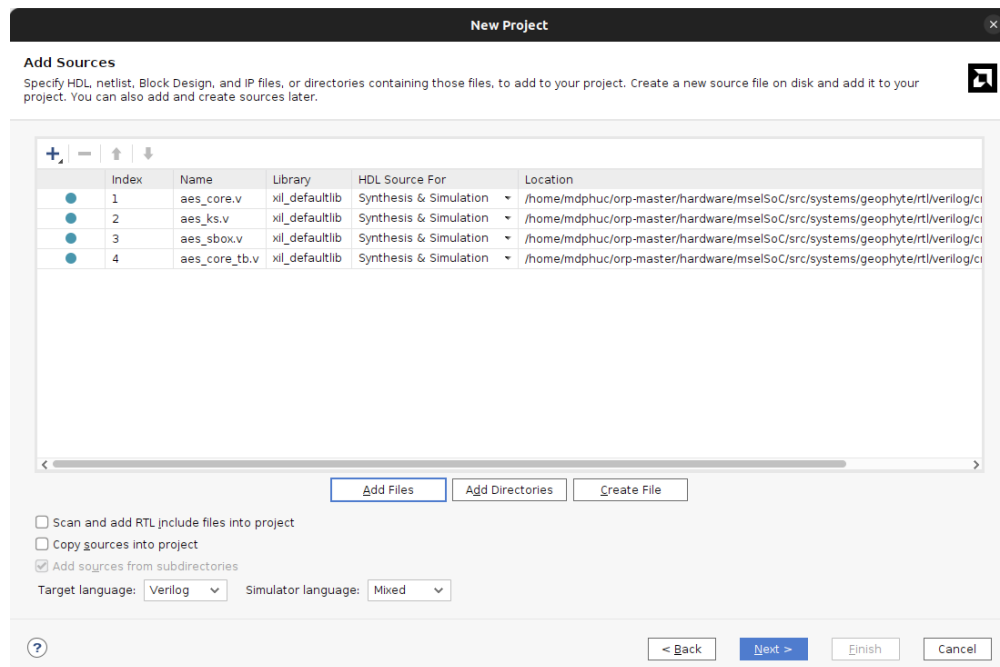


Fig. 19: Step 6: Add all the 4 files as source files in the Vivado project

- Click “Next” to proceed and add the 3 files and the testbench file to our project (as shown in Fig. 19).
- Set the Default AMD part as: `xc7a100tftg256-2` (we use ChipWhisperer CW305 FPGA board). We choose this one as we have access to CW305 FPGA board in our lab. Other parts can also be selected. Based on our experience, two different parts will lead to the same simulated

trace at the RTL level (as the RTL code remains the same) but their simulated traces will have discrepancies at the netlist level and gate level.

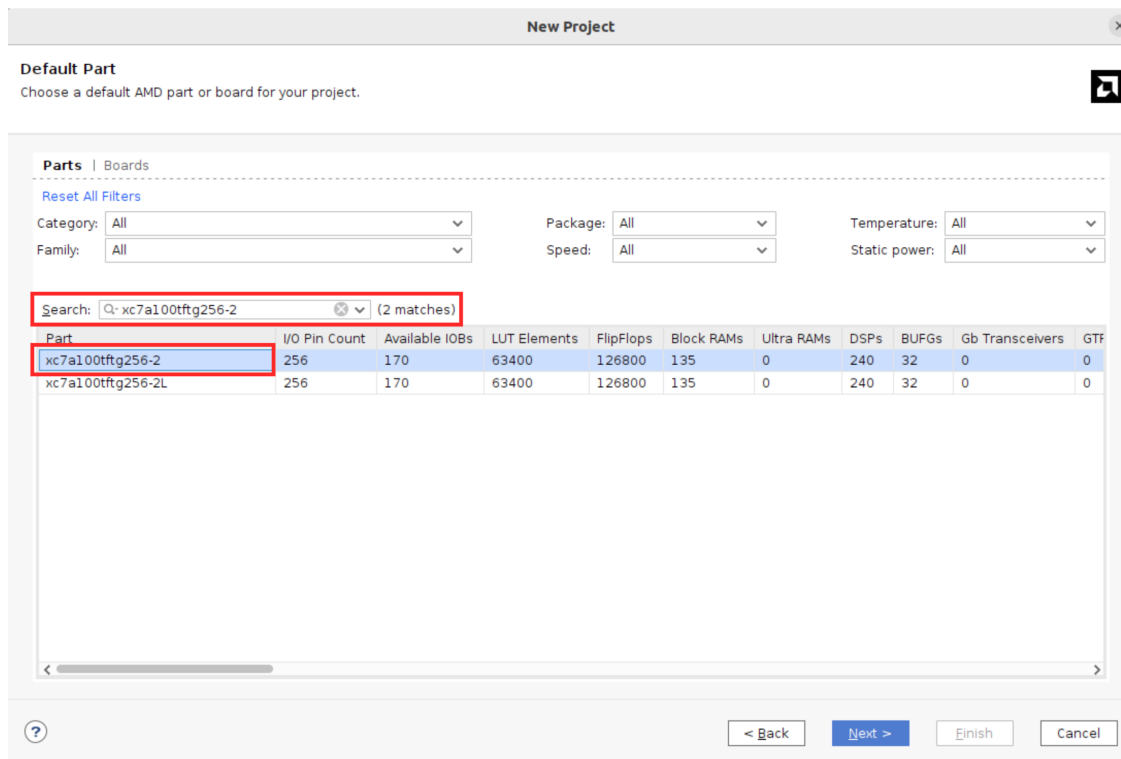


Fig. 20: Step 7: choose the default part for the project

- After loading into the project and waiting for the project to update, by default, Vivado sets `aes_core_tb.v` as the top module in the project; however, as `aes_core_tb.v` only serves for testing purpose only, it should not be a part of the design, thus cannot be the top module of the design. The top module of the design should be `aes_core.v`. To make this happen, right click on `aes_core_tb.v` and select “Move to Simulation Sources”. After that, `aes_core.v` should be selected as the top module of the design. If not, right click on “`aes_core.v`” and select “Set as Top”. This completes the initial setup for the project in Vivado.

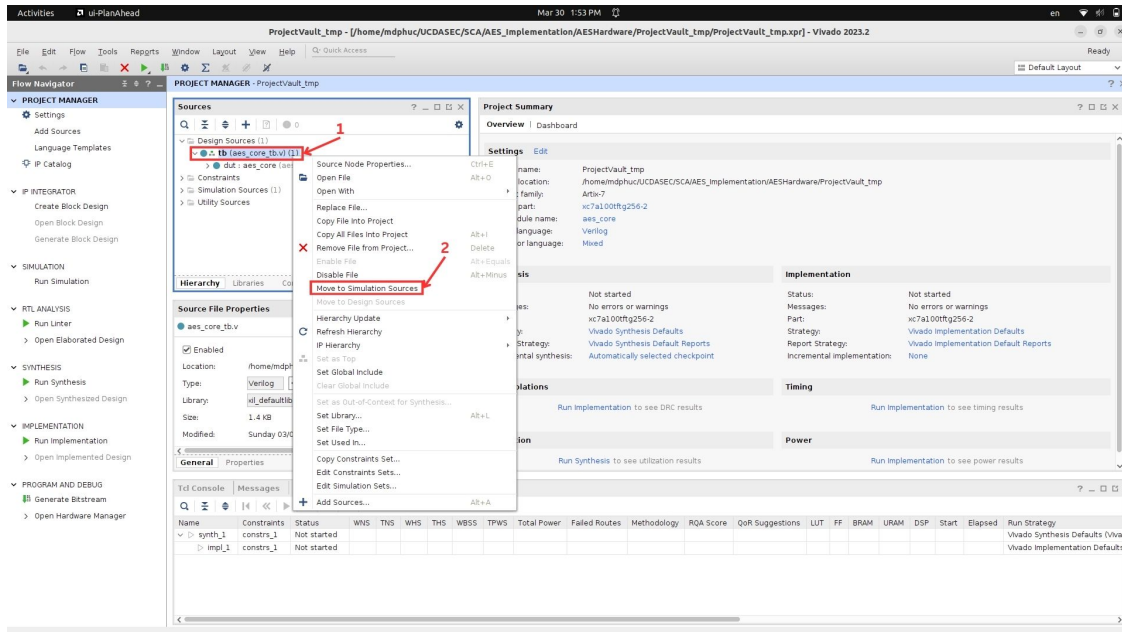


Fig. 21: Step 8: move testbench file `aes_core.tb.v` from design sources to simulation sources

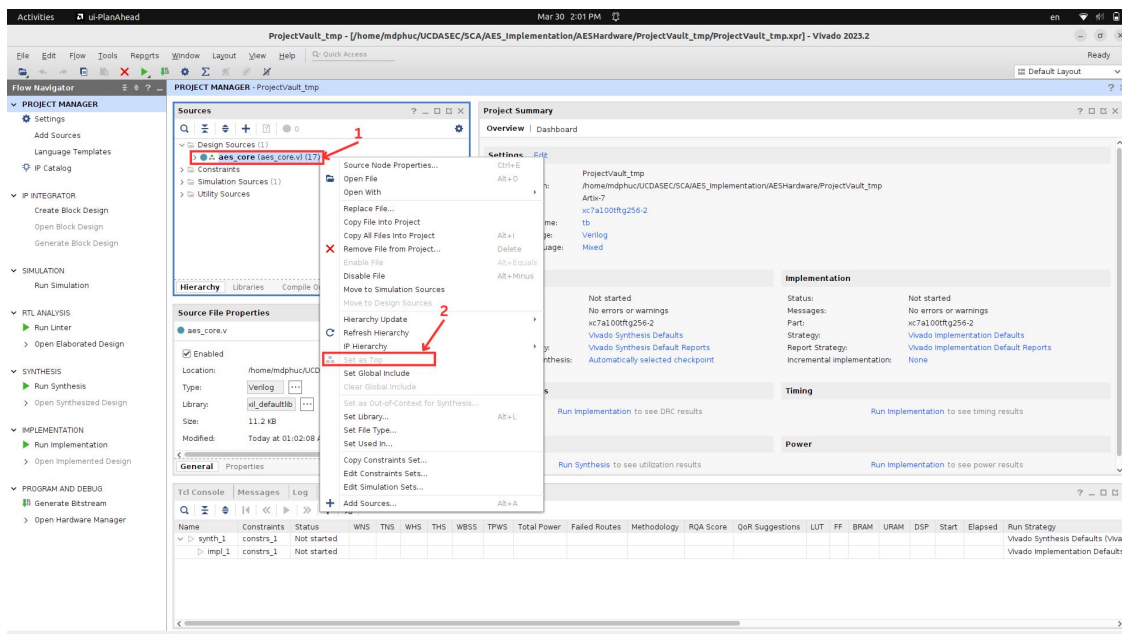


Fig. 22: Step 8: choose file `aes_core.v` as the top module (if needed)

4.2 Modify Verilog Files for the Compatibility with TOFU

At this point, typically, there are four reasons that we may need to modify the original design in Verilog (or VHDL), including (a) modifying the design so that it is synthesizable and can be implemented on the target FPGA board (e.g., CW305); (b) modifying the design so that the design

is compatible with the downstream parsing process with TOFU; (c) simplifying the design so that it runs only one instance of AES encryption; (d) adding additional lines to dump a VCD file.

For ProjectVault, we do not need to make any changes for (a) and (b) but need to make modifications for the purpose of (c) and (d). The changes for (a) vary depending on each target design. The changes for (b) generally are associated with the use of `integer`. Specifically, the use of `integer` may create unexpected behavior when parsing VCD files with TOFU later. If needed, our suggested change is to replace `integer` with `reg [31:0]` in the design source files before running the simulation. Otherwise, TOFU may have difficulties parsing the VCD file. For instance, we needed to perform this change for SecWork (unmasked AES) used in the version 1 of this tutorial.

To address (c) and (d) for ProjectVault, we make two modifications, including **(1)** simplifying `aes_core_tb.v` such that only one instance of AES-128 is executed with a chosen key (16 bytes) and plaintext (16 bytes) during one simulation as the original version encompasses test cases for both encryption and decryption; **(2)** adding VCD dump functionality in `aes_core_tb.v`

Modification I (Simplify `aes_core_tb.v`). We comment out code from line 93 to 175 in `aes_core_tb.v` as these lines are associated with AES_192, AES_256, and decryption mode of AES_128. We also add “\$finish” at line 92 so that the simulation will be stopped immediately after ProjectVault has finished its operation.

Next, we update the key (`key_in` in line 82) and plaintext (`pt` in line 83) using our chosen values: `2b7e151628aed2a6abf7158809cf4f3c` and `6bc1bee22e409f96e93d7e117393172a`, respectively. If it worths mentioning that we only need to use AES-128, so we only load the key to `key_i [255:128]` despite the register `key_i` has 256 bits in total. Next, we can comment out line 84 in `aes_core_tb.v` as we have verified the expected ciphertext with the output in advance and do not need this additional verification process for generating the VCD file. The version of `aes_core_tb.v` after **Modification I** is presented below as a reference.

```

1 // aes_core_tb.v (after modification 1)
2
3 `timescale 1ns / 1ps
4 ..... // skip lines with comments
5 module aes_core_tb;
6
7     // Inputs
8     reg clk;
9     reg load_i;
10    reg [255:0] key_i;
11    reg [127:0] data_i;
12    reg [1:0] size_i;
13    reg dec_i;
14
15    // Outputs
16    wire [127:0] data_o;
```

```

17     wire busy_o;
18
19     reg [127:0] pt;
20     reg [127:0] ct;
21
22     // Instantiate the Unit Under Test (UUT)
23     aes_core uut (
24         .clk(clk),
25         .load_i(load_i),
26         .key_i(key_i),
27         .data_i(data_i),
28         .size_i(size_i),
29         .dec_i(dec_i),
30         .data_o(data_o),
31         .busy_o(busy_o)
32     );
33
34     initial begin
35         // Initialize Inputs
36         clk = 0;
37         load_i = 0;
38         key_i = 0;
39         data_i = 0;
40         size_i = 0;
41         dec_i = 0;
42
43         // Wait 100 ns for global reset to finish
44         #100;
45
46         // Add stimulus here
47         size_i = 2'b00;
48         // Modified by us, AES-128 only so use key_i[255:128]
49         key_i[255:128] = 128'h2b7e151628aed2a6abf7158809cf4f3c;
50         // Modified by us
51         pt = 128'h6bc1bee22e409f96e93d7e117393172a;
52         // Comment out by us
53         // ct = 128'h3ad77bb40d7a3660a89ecaf32466ef97;
54
55         dec_i = 0;
56         data_i = pt;
57         load_i = 1;
58         while(!busy_o) #1;
59         load_i = 0;
60         while(busy_o) #1;
61         // Added by us
62         $finish;
63     end

```

```

64
65 always #10 clk <= ~clk;
66
67 endmodule

```

Modification II (Add VCD dumps functionality to `aes_core_tb.v`). Dumping a VCD file can be achieved from the testbench file by using the sequence of `$dumpfile` (specifying which file to dump to) and `$dumpvars` (specifying which module of the design to dump signals from). For example, one can add the following code block to `aes_core_tb.v` (after the initiation of `aes_core` module and before the `initial` block that is used to run ProjectVault (in the original testbench, it can be found on line 68).

```

1 // aes_core_tb.v
2 // .....
3 // line 69
4 initial begin
5     $dumpfile("test.vcd");
6     $dumpvars(0, uut); // dump all signals from aes_core instantiated as uut
7 end
8 // .....

```

4.3 Simulate the AES Hardware Design in Vivado

With all the modifications we mentioned in the previous subsection, we are ready to simulate the hardware design of AES-128 given a key and a plaintext and produce a VCD file using Vivado. We defer the discussions on how to generate multiple VCD files with different plaintexts in a later section.

1. In Vivado, go to “Simulation” in the “Flow Navigator” pane and click “Run Simulation” then “Run Behavioral Simulation” (as shown in Fig. 23).
2. The terminal in Vivado shows that the RTL simulation of the hardware AES-128 implementation lasts for 331 nanoseconds and the encryption runs correctly (we checked in advance). The entire simulation takes some additional time to save the VCD file. Overall, it takes about 10 seconds to complete the simulation and save one VCD file for this example. Screenshots of the simulation can be found in Fig. 24 and Fig 25.
3. Once the simulation completes, locate the generated VCD file `test.vcd` by following the path below (this depends on the name the project)

```

1 /home/username/ProjectVault_AES_Impl/ProjectVault_AES_Impl.sim/sim_1/behav/xsim
  /test.vcd

```

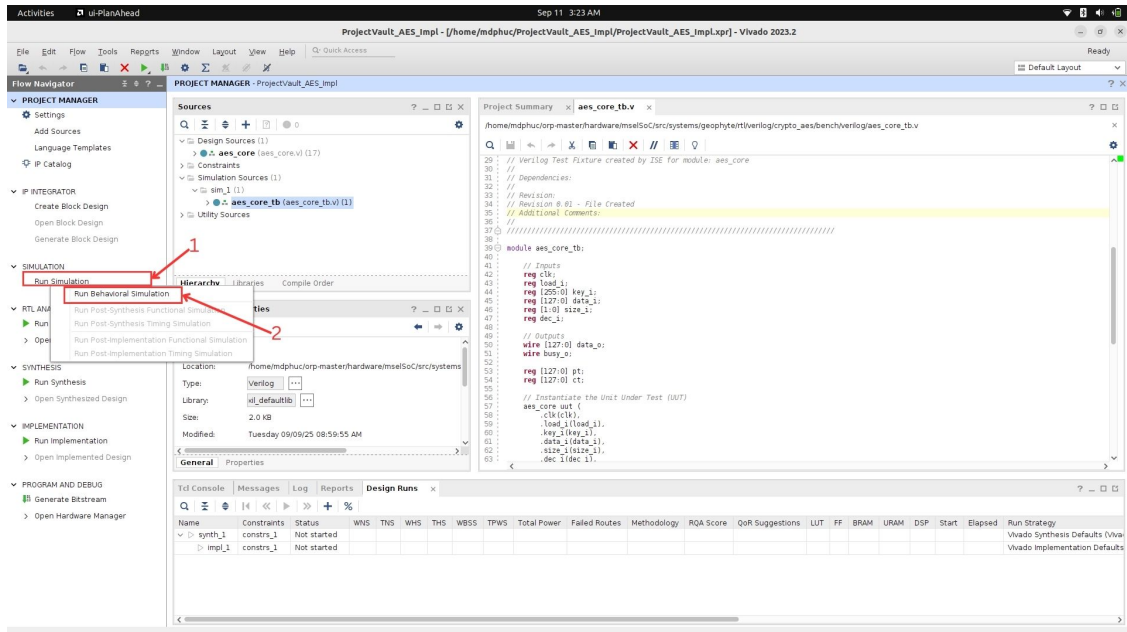


Fig. 23: Run Behavioral Simulation of the modified ProjectVault in Vivado

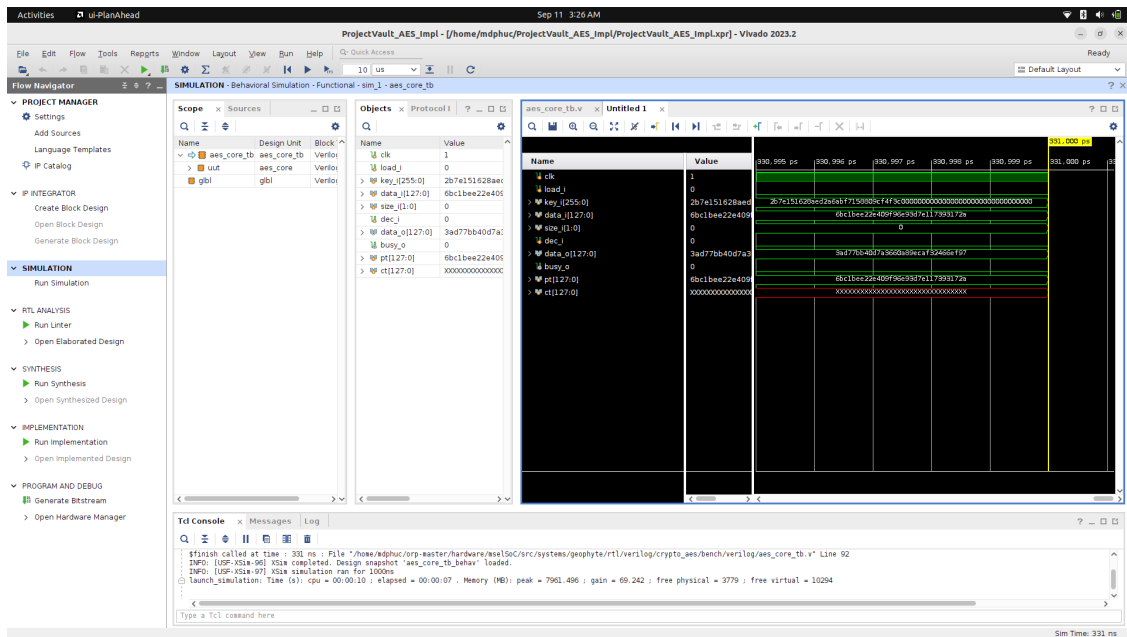


Fig. 24: One simulation given a key and a plaintext.

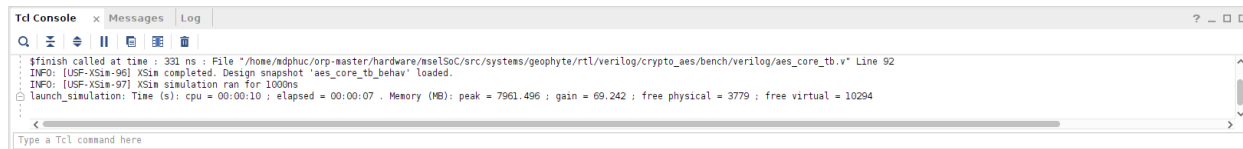


Fig. 25: The simulation time and running time of one simulation.

4.4 Post-Process a VCD File for the Parsing in TOFU

Given the VCD file from the last step, one more customized step we need to do is to remove empty lines in it. Specifically, Vivado generates a VCD file with empty lines. However, TOFU cannot parse a VCD file directly with empty lines. We write the following Python script, named `emptyline.py`, to remove empty lines in a VCD file. The same code can also be found in our repository ⁴,

```

1 #fileName: emptyline.py
2 import sys
3
4 def remove_empty_lines(file_path):
5     try:
6         with open(file_path, 'r+') as file:
7             lines = file.readlines()
8             file.seek(0)
9             file.truncate(0)
10
11             for line in lines:
12                 if line.strip():
13                     file.write(line)
14             print(f"Empty lines removed successfully in the same file: {file_path}")
15     except Exception as e:
16         print(f"An error occurred: {e}")
17
18 if __name__ == "__main__":
19     if len(sys.argv) != 2:
20         print("Usage: python script.py <file_path>")
21         sys.exit(1)
22
23     file_path = sys.argv[1]
24     remove_empty_lines(file_path)

```

One can run the script in the terminal as follows by providing the path of the original VCD file. This will remove all the empty lines in the given VCD file.

```

1 python3 emptyline.py /home/username/ProjectVault_AES_Impl/ProjectVault_AES_Impl.
   sim/sim_1/behav/xsim/test.vcd

```

⁴ https://github.com/UCdasec/SideRTL/blob/main/src_codes/emptyline.py

4.5 (Optional) Use GTKWave to view the VCD file

This step is optional. We can view the VCD file by following the command:

```
1 # run GTKWave to view a VCD file
2 gtkwave /home/username/ProjectVault_AES_Impl/ProjectVault_AES_Impl.sim/sim_1/behav
  /xsim/test.vcd
```

4.6 Generate a Simulated Trace with TOFU

After we post-process the VCD file, we are ready to use TOFU to parse it and generate a simulated trace. Specifically, we first navigate to the TOFU directory and create a working directory using the terminal:

1. Enter the `tofu` directory

```
1 cd tofu-master
```

2. Create a folder named `temp` (or a name of your choice)

```
1 mkdir temp
```

3. Copy the post-processed VCD file to folder `temp` in TOFU.

```
1 cp /home/username/ProjectVault_AES_Impl/ProjectVault_AES_Impl.sim/sim_1/behav/
  xsim/test.vcd temp/
```

4. Copy the JSON file named `settings_example.json` to folder `temp`. The JSON file is basically a configuration file for TOFU. It is originally in TOFU's examples folder.

```
1 cp example/settings_example.json temp/
```

5. Next, we open file `settings_example.json` in the `temp` directory and update the settings of TOFU as below.

- (a) set `vcdGlob` as `test.vcd` (the vcd file we generated from Vivado using ProjectVault, use the version without empty lines).
- (b) set `pickleGlob` as `test.pickle`.
- (c) set `leakageModel` as `HammingDistance` (or `HammingWeight`).
- (d) set `traceFileName` as `trace.h5` (or name of your choice).

```

{} settings_example.json X
tofu-master > temp > {} settings_example.json > ...
1 {
2   "vcdGlob": "test.vcd",
3   "pickleGlob": "test.pickle",
4   "signalsFileNameLiterals": "signals_name.json",
5   "signalsFileName": "signals.json",
6   "signalPropertiesFile": "signal_properties.pickle",
7   "leakageModel": "HammingDistance",
8   "window": false,
9   "windowFrom": null,
10  "windowTo": null,
11  "valueExtractFunction": "valueExtractIndex",
12  "writeTraces": true,
13  "writeTracesBatchSize": 10,
14  "traceFileName": "trace.h5",
15  "align": false,
16  "downsample": 1e5,
17  "format": "lascar"
18 }

```

Fig. 26: Update the settings for TOFU in `./temp/settings_example.json`.

6. Go back to `tofu-master` and run TOFU's parsing and synthesis scripts as below:

```

1 python3 parse.py --settings temp/settings_example.json
2 python3 synthesize.py --settings temp/settings_example.json

```

After completing the two scripts above, one simulated trace will be saved in HDF5 format and can be found in `./tofu-master/temp/trace.h5`. This completes the process of generating one simulated trace of AES-128 using ProjectVault.

4.7 An Example of a Simulated Trace

Given a simulated trace in `./tofu-master/temp/trace.h5`, we can generate a plot to visualize it by using the following python script

```

1 # file_name: openoneTrace.py
2 import h5py
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 with h5py.File('/home/username/tofu-master/temp/trace.h5', 'r') as file:
7
8     dataset = file['/leakages']
9 # Extracting the data from the dataset
10     data = dataset[:, :]
11

```

```

12 # Reshape the data to a 1D array
13 data = data.reshape(-1)
14 plt.plot(data)
15 plt.figure(figsize=(10, 6))
16
17 plt.tight_layout()
18 plt.show()
19 plt.savefig('/home/username/tofu-master/temp/trace.png')

```

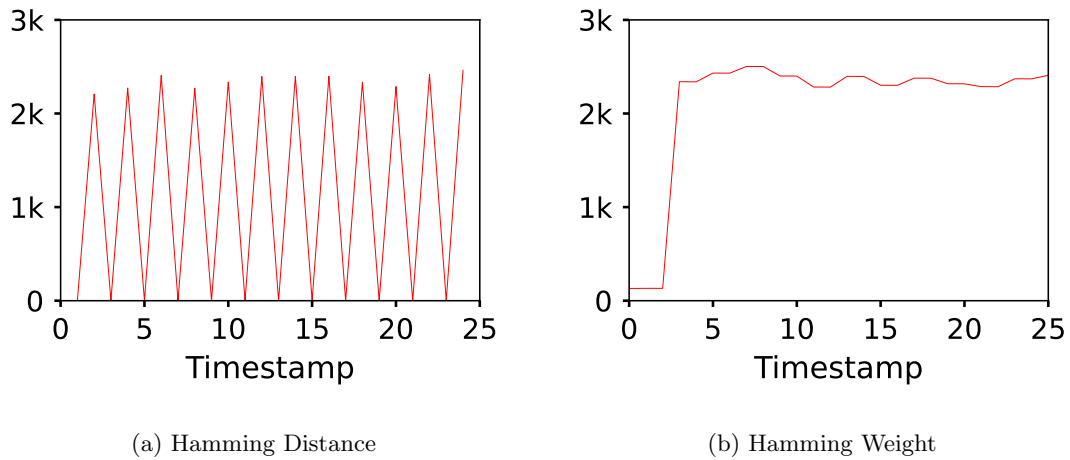


Fig. 27: Simulated traces generated by our pipeline (Vivado + ProjectVault + TOFU).

One example trace is presented in Fig. 27a. It consists of 25 samples (1 sample per 1 nanosecond in RTL simulation), where each sample is based on the toggle counts at each nanosecond from the RTL simulation in Vivado. In general, given a trace simulated with Hamming Distance model, a higher value in y-axis indicates more computation (more signal transitions) are involved at a given timestamp. Another example of a simulated trace based on Hamming Weight model is presented in Fig. 27b. The two traces from Fig. 27a and Fig. 27b were generated from the same VCD file but with a different leakage model in TOFU. The trace in Fig. 27b consists of 26 samples, one more sample compared to the number of samples in a trace produced using Hamming Distance.

It is worth mentioning that if the source code of a hardware AES implementation has been updated since the writing of this tutorial, one may observe a different simulated trace (depending on how significant the changes are).

4.8 Obtain Multiple Simulated Traces (Method Alpha)

The above description can produce one simulated trace. To obtain multiple traces, we will need to repeat the above steps by providing a new plaintext and a fixed key⁵, regenerating a VCD

⁵ One can also update the key in each simulation if desired.

4. GENERATING SIMULATED TRACES FOR UNMASKED AES

file, and reproducing a new simulated trace each time. While this can be done manually, it is not efficient as side-channel analysis typically requires a large number of traces (e.g., thousands or even millions). To generate traces automatically in a large scale (e.g., $N = 5,000$ traces), we first create a new folder, e.g., `ProjectVault_5k_HD`, in `tofu-master` (`/tofu-master/ProjectVault_5k_HD/`), in which we will save VCD files. Then, we pre-generate a `.txt` file, e.g., `plaintext5000.txt`, containing a number of N plaintexts in hexadecimal, one for each line (the first few lines of the `.txt` file are shown below). Then, we write a script to run the simulation in Vivado by updating the plaintext in `aes_core_tb.v` and the name of the VCD file each time. Running the script will result N VCD files. The code of this script can be found in our repo ⁶.

```

1 # plaintext5000.txt
2 74698c881ce1d6e3561dce317ffa294d
3 2b38e6fa249a844b3ade9983f7709ed3
4 9819894f58bb4f270b7eb5f43b43f1c8
5 6efba58de7a76a52de9400bd2e612d57
6 .....

1 # file_name: run_vivado.sh
2 #!/bin/bash
3
4 VIVADO_PATH="/home/username/Vivado/2023.2"
5 source "$VIVADO_PATH/settings64.sh"
6
7 # Navigate to the project directory
8 cd /home/username/ProjectVault_AES_Impl
9
10 # path of aes_core_tb.v
11 file_path="/home/username/ProjectVault_AES_Impl/ProjectVault_AES_Impl.srscs/
    sources_1/imports/src/tb/aes_core_tb.v"
12 # the line of plaintext in aes_core_tb.v
13 line_number=85
14 # path of a file with pre-generated plaintexts, one plaintext per line.
15 plaintext_file="/home/username/Plaintexts/plaintexts5000.txt"
16 # the total number of N traces
17 iterations=5000
18
19 for i in $(seq 1 $iterations); do
20     # update the plaintext at line_number in aes_core_tb.v
21     new_content=$(sed -n "${i}p" "$plaintext_file")
22     new_line="000000pt_128'h$new_content;"
23     sed -i "${line_number}s/./$new_line/" "$file_path"
24     echo "Replaced line $line_number with: $new_line"
25
26     # update the VCD file name at line 71 in aes_core_tb.v

```

⁶ https://github.com/UCdasec/HardSide/blob/main/bash_scripts/run_vivado.sh

```

27 new_dumpfile="/home/username/tofu-master/ProjectVault_5k_HD/plaintext$((0 +
    i)).vcd\"
28 sed -i "71s/.*/\_\_\_\_\$dumpfile($new_dumpfile);/" "$file_path"
29 echo "Changed_line_71_to:\_\$dumpfile($new_dumpfile);"
30
31 # Run simulations by calling another script launch_simulation.tcl
32 vivado -mode tcl -source launch_simulation.tcl
33 done

```

The bash script above changes the plaintext and the name of the VCD file every loop. The `launch_simulation.tcl`⁷ below is responsible to run the simulation each time after the changes of the plaintext.

```

1 # file name : launch_simulation.tcl
2 open_project ProjectVault_AES_Impl.xpr
3 launch_simulation
4 close_sim
5 exit

```

It is worth mentioning that the overall time to generate a large number of simulated traces can be time-consuming and increases linearly with the number of simulated traces. Specifically, it takes about 10 seconds to obtain one VCD file from one RTL simulation given a key and a plaintext. In addition, it takes about another 10 seconds to exit Vivado and restart the simulation for every new plaintext (at least given the current version). For instance, to obtain 5,000 VCD files, it takes about **2.5 days** to complete all the RTL simulations. In addition, the storage requirement is also high. Given 5,000 VCD files from ProjectVault, it takes about **1 GB** for storage.

Once we have a large number of VCD files, we write another script `emptyline.sh`⁸ to automatically remove empty lines in all these VCD files.

```

1 # emptyline.sh
2 #!/bin/bash
3
4 # Initial setup for processing 5000 VCD files
5 for ((i=1; i<5000; i++))
6 do
7     current_filename="plaintext$i"
8     # Run emptyline.py
9     python3 /home/username/emptyline.py /home/username/tofu-master/
        ProjectVault_5k_HD/$current_filename.vcd
10 done

```

⁷ https://github.com/UCdasec/HardSide/blob/main/bash_scripts/launch_simulation.tcl

⁸ https://github.com/UCdasec/HardSide/blob/main/bash_scripts/emptyline.sh

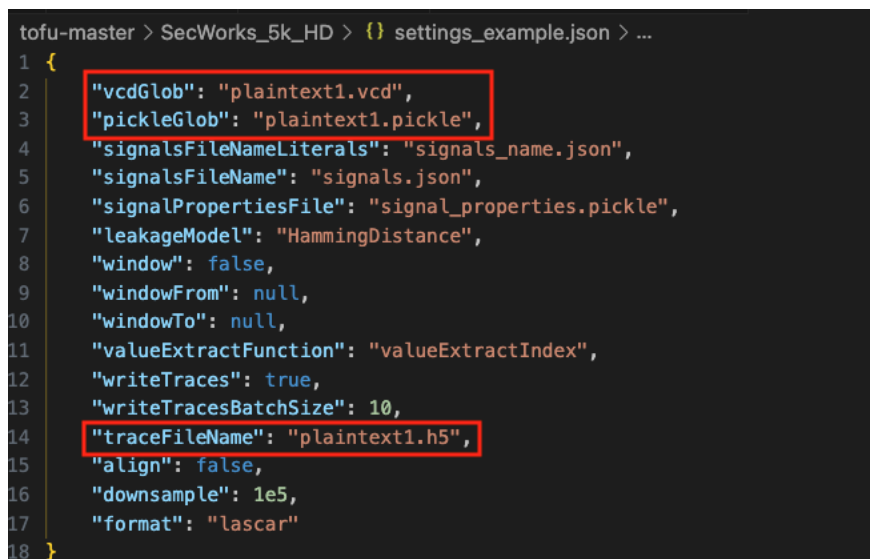
Given all the post-processed VCD files (without empty lines), we can run another script `parseTOFU.sh`⁹ to produce the corresponding simulated traces automatically. But before running the script `parseTOFU.sh`,

- We copy the `settings.json` file from `/tofu-master/temp` to a new folder named `/tofu-master/ProjectVault_5k_HD`, where we have all the post-processed VCD files. Instead, we can also use the following command:

```
1 cp /home/username/tofu-master/temp/setting_example.json /home/username/tofu-
  master/ProjectVault_5k_HD/
```

- We modify the `settings.json` file to ensure that the `vcdGlob`, `pickleGlob`, and `traceFileName` fields are all set to `plaintext1.vcd`, `plaintext1.pickle`, `plaintext1.h5` (as shown in Fig. 28). This guarantees consistency in naming when starting the trace generation process.

Once the above steps are completed, we run `parseTOFU.sh`. After that we obtain a corresponding number of N `.h5` files, one for each simulated trace we generate (e.g., `plaintext1.h5` for the first plaintext).



```
tofu-master > SecWorks_5k_HD > {} settings_example.json > ...
1 {
2   "vcdGlob": "plaintext1.vcd",
3   "pickleGlob": "plaintext1.pickle",
4   "signalsFileNameLiterals": "signals_name.json",
5   "signalsFileName": "signals.json",
6   "signalPropertiesFile": "signal_properties.pickle",
7   "leakageModel": "HammingDistance",
8   "window": false,
9   "windowFrom": null,
10  "windowTo": null,
11  "valueExtractFunction": "valueExtractIndex",
12  "writeTraces": true,
13  "writeTracesBatchSize": 10,
14  "traceFileName": "plaintext1.h5",
15  "align": false,
16  "downsample": 1e5,
17  "format": "lascar"
18 }
```

Fig. 28: Edit the `./tofu-master/ProjectVault_5k_HD/settings_example.json` file to start generating multiple VCD files

```
1 # parseTOFU.sh
2 #!/bin/bash
3
4 # Initial setup for parsing 5000 post-processed VCD files
5 for ((i=0; i<5000; i++))
```

⁹ https://github.com/UCdasec/HardSide/blob/main/bash_scripts/parseTOFU.sh

```

6 do
7     current_filename="plaintext$i"
8     next_filename="plaintext$((i+1))"
9
10 # Modify JSON file settings_example.json each time
11     json_file="/home/username/tofu-master/ProjectVault_5k_HD/settings_example.json"
12     sed -i "s/\\"vcdGlob\\":_\\\\"$current_filename.vcd\\",/\\"vcdGlob\\":_\\\\"$next_filename.vcd\\",/" "$json_file"
13     sed -i "s/\\"pickleGlob\\":_\\\\"$current_filename.pickle\\",/\\"pickleGlob\\":_\\\\"$next_filename.pickle\\",/" "$json_file"
14     sed -i "s/\\"traceFileName\\":_\\\\"$current_filename.h5\\",/\\"traceFileName\\":_\\\\"$next_filename.h5\\",/" "$json_file"
15
16     cd /home/username/tofu-master
17
18     python3 parse.py --settings ProjectVault_5k_HD/settings_example.json
19
20     python3 synthesize.py --settings ProjectVault_5k_HD/settings_example.json
21
22 done

```

Removing empty lines in these VCD files and executing TOFU to obtain these simulated traces is less time-consuming. For instance, to parse 5,000 VCD files to generate 5,000 simulated traces, it takes about 50 minutes. On the other hand, the storage requirement remains high. Given 5,000 VCD files from ProjectVault to parse and synthesize using TOFU to 5,000 H5 files, it takes about **2.7 GBs** for storage.

Merge Traces into a Dataset. Given all the traces we generated in h5 format, the plaintexts in the .txt file, and the key we used in the simulation, we can now create a dataset in .npz format, where each tuple in the dataset is represented as (trace, plaintext, key). For the ease of later analysis, we save each byte in the plaintext/key as an integer in decimal rather than in hexadecimal. We use a python script to first convert the plaintexts in plaintext5000.txt to the ones in plaintext5000_decimal.txt. The first few lines of plaintext5000_decimal.txt are shown below as a reference.

```

1 # python script from hex to demical
2 INPUT_FILE = "/home/username/plaintext5000.txt"
3 OUTPUT_FILE = "/home/username/plaintext5000_decimal.txt"
4
5 with open(INPUT_FILE, "r", encoding="utf-8") as fin, \
6     open(OUTPUT_FILE, "w", encoding="utf-8") as fout:
7
8     for line in fin:
9         hex_str = line.strip()
10        if not hex_str:

```

```

11         continue
12
13         dec_line = "".join(str(int(hex_str[i:i+2], 16)) for i in range(0, 32, 2))
14         fout.write(dec_line + "\n")
15
16 print(f"Done! Wrote decimal plaintexts to '{OUTPUT_FILE}'.")

```

```

1 #plaintext5000_decimal.txt
2 116 105 140 136 28 225 214 227 86 29 206 49 127 250 41 77
3 43 56 230 250 36 154 132 75 58 222 153 131 247 112 158 211
4 152 25 137 79 88 187 79 39 11 126 181 244 59 67 241 200
5 110 251 165 141 231 167 106 82 222 148 0 189 46 97 45 87
6 .....

```

We then run the following script (`saveNPZ.py`¹⁰) to create the dataset.

```

1 #saveNPZ.py
2 import h5py
3 import os
4 import numpy as np
5 output_path = '/home/username/ProjectVault_HD_NPZ/'
6 os.makedirs(output_path, exist_ok=True)
7
8 file_name = 'ProjectVault_5000_HD.npz'
9
10 outpath = os.path.join(output_path, file_name)
11
12 text_file_path = '/home/username/Plaintexts/plaintext5000_decimal.txt'
13 with open(text_file_path, 'r') as file:
14     plaintexts = file.readlines()
15 plain_text = np.array([list(map(int, line.split())) for line in plaintexts], dtype
16                       = np.uint16)
17
18 # key used in the simulation for generating all the VCD files.
19 static_key = [210, 213, 1, 104, 130, 131, 145, 67, 150, 158, 233, 162, 83, 167,
20               82, 225]
21 key = np.array(static_key, dtype=np.uint8)
22
23 # the path where all the h5 files are stored.
24 file_pattern = 'home/username/tofu-master/ProjectVault_5k_HD/plaintext{}.h5'
25 all_trace_arrays = []
26 for i in range(1, 4):
27     file_path = file_pattern.format(i)
28     with h5py.File(file_path, 'r') as h5_file:
29         trace_array = h5_file['leakages'][:]
30         all_trace_arrays.append(trace_array)

```

¹⁰ https://github.com/UCdasec/HardSide/blob/main/python_scripts/saveNPZ.py

```

29 power_trace = np.concatenate(all_trace_arrays , axis=0)
30
31 print(f"Shape_of_Power_Trace:{np.shape(power_trace)}")
32 print(power_trace)
33 print(f"Shape_of_Plaintexts:{np.shape(plain_text)}")
34 print(plain_text)
35 print(f"Shape_of_Key:{np.shape(key)}")
36 print(key)
37
38 np.savez(outpath, power_trace=power_trace, plain_text=plain_text, key=key)

```

4.9 Obtain Multiple Simulated Traces (Method Beta)

Our above approach (Version Alpha) to automatically generate simulated traces works, but it is time-consuming, mainly due to the exit and restart of Vivado per simulation. To address this limitation, we improve our approach (named Version Beta), which does not need to exit or restart Vivado per trace. In other words, instead of running `launch_simulation.tcl` N times in version alpha, we only need to run `launch_simulation.tcl` one time in version beta. To achieve this, we need to first modify additional lines in the testbench file (`aes_core_tb.v`).

Additional Modification in `aes_core_tb.v`. To prevent Vivado from exiting and restarting from each simulation given a different plaintext, we add an I/O mechanism in the testbench file `aes_core_tb.v` to automatically load the plaintext from an additional `config.text` file, which is located in the same directory as the Vivado project. Specifically, we create an additional task block named `loadConfig()` in `aes_core_tb.v` to handle this task. This block also automatically updates the VCD file name accordingly. Three additional registers, including `fileHandler`, `plaintextNum`, and `vcdFilename`, are created correspondingly to support this block. Finally, we add an additional line to call this `loadConfig()` task after we load the key. The main changes in `aes_core_tb.v` are highlighted below:

```

1  // aes_core_tb.v (our modified version, version beta)
2  `timescale 1ns / 1ps
3  ..... // skip comment lines
4  module aes_core_tb;
5
6      ..... // remain the same
7
8      // add additional registers (version 2)
9      reg [63:0] fileHandler;
10     reg [31:0] plaintextNum;
11     reg [25*8:0] vcdFilename;
12
13     // Instantiate the Unit Under Test (UUT)
14     aes_core uut (
15         ..... // remain the same

```

```

16     );
17
18     // new task, loading plaintext and updating VCD file name through I/O
19     task loadConfig;
20         begin
21             fileHandler = $fopen("config.txt","r");
22             if (fileHandler) begin
23                 $fscanf(fileHandler, "%x,%d", pt, plaintextNum);
24                 $fclose(fileHandler);
25             end
26             $sformat(vcdFilename, "plaintext%0d.vcd", plaintextNum);
27             $dumpfile(vcdFilename);
28             $dumpvars(0, uut);
29             $display("Dumping to: %s", vcdFilename);
30         end
31     endtask
32
33     initial begin
34         // Initialize Inputs
35         clk = 0;
36         load_i = 0;
37         key_i = 0;
38         data_i = 0;
39         size_i = 0;
40         dec_i = 0;
41
42         // Wait 100 ns for global reset to finish
43         #100;
44
45         // Add stimulus here
46         size_i = 2'b00;
47         key_i[255:128] = 128'h2b7e151628aed2a6abf7158809cf4f3c;
48
49         // Load plaintext from config.txt file (added in version beta)
50         loadConfig();
51
52         dec_i = 0;
53         data_i = pt;
54         load_i = 1;
55         while(!busy_o) #1;
56         load_i = 0;
57         while(busy_o) #1;
58         $finish;
59     end
60
61     always #10 clk <= ~clk;
62

```

```
63 endmodule
```

An example of the config.txt file is presented below as a reference, where 74698c881ce1d6-e3561dce317ffa294d is a plaintext and 0 is the index of 74698c881ce1d6e3561dce317ffa294d in the plaintext file

```
1 74698c881ce1d6e3561dce317ffa294d , 0
```

4.10 All in One Script for Method Beta

To automate the entire process of the method beta, we also create an all-in-one Python script, named `vivado_all_in_one_tofu_capture.py`, which allows us to run the entire process without running the scripts from each step separately. This all-in-one script interacts with Vivado to run the simulation, processes the traces with TOFU, and creates the final dataset automatically. The full Python script can be found in our repository on GitHub [1]

In this section, we provide an example of the key parameters that we need to set in this all-in-one script before we run it.

```
1 # vivado_all_in_one_tofu_capture.py (parameters)
2 SAVED_TRACE_BASE_DIRECTORY = r"/home/mdphuc/UCDASEC/SCA/AES_Implementation/
   AESHardware/ProjectVault" # Final folder will be here
3 VIVADO_DIRECTORY           = r"/tools/Xilinx/Vivado/2023.2"
4 VIVADO_PROJECT_DIRECTORY   = r"/home/mdphuc/UCDASEC/SCA/AES_Implementation/
   AESHardware/ProjectVault/ProjectVault_Vivado"
5 VIVADO_PROJECT_XPR_PATH     = r"/home/mdphuc/UCDASEC/SCA/AES_Implementation/
   AESHardware/ProjectVault/ProjectVault_Vivado/ProjectVault_Vivado.xpr"
6 VIVADO_PROJECT_SIM_BASE     = r"/home/mdphuc/UCDASEC/SCA/AES_Implementation/
   AESHardware/ProjectVault/ProjectVault_Vivado/ProjectVault_Vivado.sim/sim_1"
7 SIMULATION_MODE             = "POST_SYNTHESIS" # RTL, POST_SYNTHESIS, or
   POST_IMPLEMENTATION
8 SIMULATION_TYPE             = "FUNCTIONAL" # TIMING or FUNCTIONAL, only matters if
   SIMULATION_MODE is not RTL
9 PLAINTEXT_FILE_PATH         = r"/home/mdphuc/opt/50000_plaintexts.txt"
10 TOFU_DIRECTORY             = r"/home/mdphuc/opt/tofu"
11 TOFU_MODE                   = r"HammingWeight" # HammingDistance or HammingWeight
12 FIXED_KEY                   = [43, 126, 21, 22, 40, 174, 210, 166, 171, 247, 21,
   136, 9, 207, 79, 60]
```

The output directory is provided to `SAVED_TRACE_BASE_DIRECTORY`, where an additional folder will be created to contain the final .npz dataset and all intermediary files (such as VCD files). The Vivado directory parameter `VIVADO_DIRECTORY` (which contains bin, lib, etc. directories), should be pointed towards the Vivado folder. Several parameters, `VIVADO_PROJECT_DIRECTORY`, `VIVADO_PROJECT_XPR_PATH`, `VIVADO_PROJECT_SIM_PATH` are needed to specify dynamic Vivado project constants, including the Vivado project folder, XPR file, and the simulation folder. The combination of `SIMULATION_MODE` and `SIMULATION_TYPE` are needed to instruct Vivado to run simulation

at what level, for example, `SIMULATION_MODE = POST_SYNTHESIS` and `SIMULATION_TYPE = FUNCTIONAL` mean that simulation will be run at Post-Synthesis Functional level. The `PLAINTEXT_FILE_PATH` contains all the N plaintexts that we will need to use for the simulation. The number of plaintexts in this .txt file also decides the simulated traces that we will generate (i.e., one plaintext for one simulated trace). All the plaintexts are saved in the hexadecimal format with one plaintext per line.

```

1 # plaintext5000.txt
2 74698c881ce1d6e3561dce317ffa294d
3 2b38e6fa249a844b3ade9983f7709ed3
4 9819894f58bb4f270b7eb5f43b43f1c8
5 6efba58de7a76a52de9400bd2e612d57
6 .....

```

Next, TOFU parameters are set including the location of the TOFU directory `TOFU_DIRECTORY` and the mode of TOFU trace generation `TOFU_MODE`. Finally, the fixed key (in the decimal format) that is used for all encryptions is set by `FIXED_KEY`. This is the same key (in the hexadecimal format) that is set within the testbench file `aes_core_tb.v`. After settings these parameters, running the script will result N VCD files automatically.

What Happens After We Run Scripts. Once the above parameters are set, we can run the all-in-one Python script (`python vivado_all_in_one_tofu_capture.py`). We explain the details after running the script to help readers understand the process behind the scene (in case any customizations that readers would like to perform). Specifically, the following script first generates two temporary script files that are used to launch Vivado and run simulations automatically.

```

1 # vivado_all_in_one_tofu_capture.py (part)
2 def generateStartScript(startScriptPath, tclScriptName):
3     scriptContents = [
4         '#!/bin/bash',
5         f'VIVADO_PATH="{VIVADO_DIRECTORY}"',
6         'source "$VIVADO_PATH/settings64.sh"',
7         f'vivado -mode tcl -source "{tclScriptName}"'
8     ]
9     with open(startScriptPath, 'w') as script:
10         for line in scriptContents:
11             script.write(f"{line}\n")
12
13 def generateTCLScript(tempTCLScriptName):
14     scriptContents = [
15         f'open_project {VIVADO_PROJECT_XPR_PATH}',
16         'launch_simulation',
17         f'set plaintexts "{PLAINTEXT_FILE_PATH}"',
18         f'set config_file "{os.path.join(VIVADO_PROJECT_SIM_PATH, "config.txt")}"',
19         ',
20         'set fh [open $plaintexts_r]',

```

```

20     r'set_plaintexts [split [read $fh] "\n"]',
21     'close $fh',
22     'set_iteration 1',
23     'foreach_plaintext $plaintexts {',
24     '        set_cfg_fh [open $config_file w]',
25     '        puts $cfg_fh [format "%s,%d" $plaintext $iteration]',
26     '        close $cfg_fh',
27     '        restart',
28     '        run_all',
29     '        puts "Iteration $iteration"',
30     '        incr_iteration',
31     '}',
32     'exit'
33 ]
34 with open(tempTCLScriptName, 'w') as script:
35     for line in scriptContents:
36         script.write(f"{line}\n")

```

A starting bash script is generated from the `generateStartScript` function. The purpose of this script is to simply start Vivado using the user's settings file `settings64.sh` from the Vivado base directory. Vivado then runs the TCL script, which is also automatically generated using the `generateTCLScript` function. The TCL script is important to the efficient generation of VCD files, as it controls shared variables between the TCL script and Verilog code. Rather than having to resynthesize the design each time the plaintext is changed, the TCL script instead sets the `$plaintext` value for each run of the simulation. These plaintext values are retrieved from the `PLAINTEXT_FILE_PATH` set at the start of the script. An extra iterator variable, `$iteration` is also shared between the script and Verilog file to change the output name of the VCD file. These two files are set through an intermediate `config.txt` file. The `config.txt` file consists of one line with a plaintext and a counter, and this line is rewritten by the all-in-one script for each simulation. A snapshot of the content of this `config.txt` using the 10th plaintext is presented below.

```

1 # config.txt
2 496af9ae847dcba6319c131e1357cf01,10

```

34 As a reminder, the TCL script requires the special `loadConfig()` task within the testbench file `aes_core_tb.v` mentioned in method beta in order to make updates in the `config.txt`. When generating 5,000 traces, the time to complete the Vivado simulation and dumping VCD files (with the method beta) took around **4 hours** on an 8-core Ubuntu VM with (either using separate scripts or this all-in-one script).

Afterward, each generated VCD file is post-processed for further TOFU processing. Empty lines in the generated VCD files must be removed as mentioned before. The following Python function is run on each VCD file and takes negligible time:

```

1 # vivado_all_in_one_tofu_capture.py (part)

```

```

2 def removeEmptyLinesOneFile(filepath):
3     with open(filepath, "r+") as vcd:
4         lines = vcd.readlines()
5         vcd.seek(0)
6         vcd.truncate(0)
7         for line in lines:
8             if line.strip():
9                 vcd.write(line)

```

The VCD files are now ready to be parsed by the TOFU tool to create simulated trace files. In the current version of our script, we must prepare the settings JSON file utilized by TOFU for each VCD file. This settings JSON file is automatically generated by the `generateTOFUSettings` function:

```

1 # vivado_all_in_one_tofu_capture.py (part)
2 def generateTOFUSettings(tofuSettingsFilePath, traceName):
3     jsonContents = [
4         '{',
5         f'      "vcdGlob": "{traceName}.vcd",',
6         f'      "pickleGlob": "{traceName}.pickle",',
7         '      "signalsFileNameLiterals": "signals_name.json",',
8         '      "signalsFileName": "signals.json",',
9         f'      "signalPropertiesFile": "signal_properties.pickle",',
10        f'      "leakageModel": "{TOFU_MODE}",',
11        '      "window": false,',
12        '      "windowFrom": null,',
13        '      "windowTo": null,',
14        '      "valueExtractFunction": "valueExtractIndex",',
15        '      "writeTraces": true,',
16        '      "writeTracesBatchSize": 10,',
17        f'      "traceFileName": "{traceName}.h5",',
18        '      "align": false,',
19        '      "downsample": 1e5,',
20        '      "format": "lascar"',
21        '    ],
22
23    with open(tofuSettingsFilePath, 'w') as jsonFile:
24        for line in jsonContents:
25            jsonFile.write(f"{line}\n")

```

The `traceName` variable corresponds with the output VCD filename, and was collected during Vivado simulation in ascending order. TOFU's `parse.py` and then `synthesize.py` are run next for each VCD file:

```

1 # vivado_all_in_one_tofu_capture.py (part)
2 currentSettingsFile = None
3 for vcdFile in tqdm(vcdFileList):

```

```

4     _, plaintextFileName = os.path.split(vcdFile)
5     currentSettingsFile = os.path.join(intermediateFiles, "settings_example.
        json")
6     generateTOFUSettings(currentSettingsFile, plaintextFileName[:-4])
7     subprocess.run([f'python3 "{os.path.join(TOFU_DIRECTORY, "parse.py")}" --
        settings "{currentSettingsFile}"'], shell=True)
8     subprocess.run([f'python3 "{os.path.join(TOFU_DIRECTORY, "synthesize.py")}"
        --settings "{currentSettingsFile}"'], shell=True)

```

Each VCD file must be parsed by TOFU to create an intermediary pickle data file, which is then synthesized into a h5 file containing exactly one trace. Given N plaintexts, a total number of N h5 files will be generated. Note that TOFU simulated trace generation represents the most time commitment in the entire trace generation process. For 50,000 traces, the TOFU step took around **30 hours** on an 8-core Ubuntu VM.

After all intermediary pickle and H5 files have been processed by TOFU, all the N h5 trace files are merged into one .npz dataset with the plaintexts and the key. One additional step of processing is needed to convert the plaintexts in hexadecimal in `plaintext50000.txt` into the decimal format standard for the .npz dataset. The entire merging process takes a negligible amount of time:

```

1  # vivado_all_in_one_tofu_capture.py (part)
2  key = np.array(FIXED_KEY, dtype=np.uint8)
3
4  # Extract plaintexts
5  decimalPlaintexts = list()
6  with open(PLAINTEXT_FILE_PATH, 'r') as hexPlaintexts:
7      plaintexts = hexPlaintexts.readlines()
8      for plaintext in plaintexts:
9          decimalPlaintext = list()
10         strippedPlaintext = plaintext.strip()
11         for i in range(0, len(strippedPlaintext), 2):
12             decimalByte = int(strippedPlaintext[i:i+2], 16)
13             decimalPlaintext.append(decimalByte)
14         decimalPlaintexts.append(decimalPlaintext)
15  plain_text = np.array(decimalPlaintexts, dtype=np.uint8)
16
17  # Extract traces
18  h5FilePaths = list()
19  for vcdFileName in vcdFileList:
20      h5FilePaths.append("".join([vcdFileName[:-4], ".h5"]))
21  traces = list()
22  for h5FilePath in h5FilePaths:
23      with h5py.File(h5FilePath, 'r') as h5File:
24          trace = h5File['leakages'][:]
25          traces.append(trace)
26  power_trace = np.concatenate(traces, axis=0)

```

```

27     outputTraceFile = os.path.join(outputDir, f"Synthetic{TOFU_MODE}_K1_{int((len(
        traces)/1000)+0.5)}k")
28     np.savez(outputTraceFile, power_trace=power_trace, plain_text=plain_text, key=
        key)

```

At this point, the entire simulated trace creation process has been completed. One extra step is to compress the intermediary files used to create the final dataset to greatly save on final disk usage. Around **27 GB** of intermediary files are created during the creation of a 50,000 trace dataset, but compressing the intermediate files will result in only **9 GB** of disk space. The compression step takes around **10 seconds** for 50,000 traces:

```

1  # the name of this script
2      with zipfile.ZipFile(os.path.join(outputDir, "Intermediate_Files.zip"), "w",
        zipfile.ZIP_DEFLATED, allowZip64=True) as zipFile:
3          for item in os.listdir(intermediateFiles):
4              itemPath = os.path.join(intermediateFiles, item)
5              if os.path.isfile(itemPath):
6                  zipFile.write(itemPath)
7      subprocess.run([f'rm -rf {intermediateFiles}'], shell=True)

```

Note that the intermediary files (e.g., VCD files) can also be deleted after the final dataset is created (but we highly recommend to keep the VCD files if possible). The final .npz dataset with 50,000 simulated traces is only around **6 MB**. Additional files included in the final export directory are a picture of one resultant trace and a reproducibility text report containing the settings used for the script, which take up a negligible amount of space.

Since Hamming Distance and Hamming Weight in TOFU can use the same set of VCD files, it is worth mentioning that we can also run the (partial) all-in-one script (assuming that VCD files have been produced) if we would like to generate simulated traces of Hamming Weight on the same design without the need of regenerating the VCD files. We skip the details here and the partial all-in-one script can be found in our repository.

4.11 Two Example Datasets (ProjectVault, RTL Simulation)

Two example datasets, ProjectVault_HD_K1_50k and ProjectVault_HW_K1_50k are generated by following the above steps (either approach alpha or approach beta). Each dataset consists of 50,000 traces. They are generated from the same set of 50,000 VCD files (a fixed key and 50,000 plaintexts from ProjectVault). The two datasets can be accessed via [4]. Details regarding how to perform side-channel analysis and the results from side-channel analysis over these two example datasets can be found in another document [4].

4.12 Simulated Traces from Netlist Level and Gate Level

In Vivado, we can also run the simulations at the netlist level by running Post-Synthesis Simulation and the ones at the gate level by running Post-Implementation Simulation. We can also

collect traces at both levels in addition to the ones that we can capture at RTL. No additional changes are needed in the design, testbench file, and our scripts (excepting running a different `tcl` command for each level of simulation).

Specifically, we previously needed to run the following `tcl` command to run RTL simulation

```
1 # file name : launch_simulation.tcl (RTL)
2 open_project ProjectVault_AES_Impl.xpr
3 launch_simulation
4 close_sim
5 exit
```

For post-synthesis (timing) simulation, we can run the following commands instead

```
1 # file name : launch_simulation.tcl (Post-Synthesis, Timing)
2 open_project ProjectVault_AES_Impl.xpr
3 open_run sythn_1
4 launch_simulation -mode post-synthesis -type TIMING
5 close_sim
6 exit
```

If it is functional, we can replace `TIMING` with `FUNCTIONAL` for `-type`.

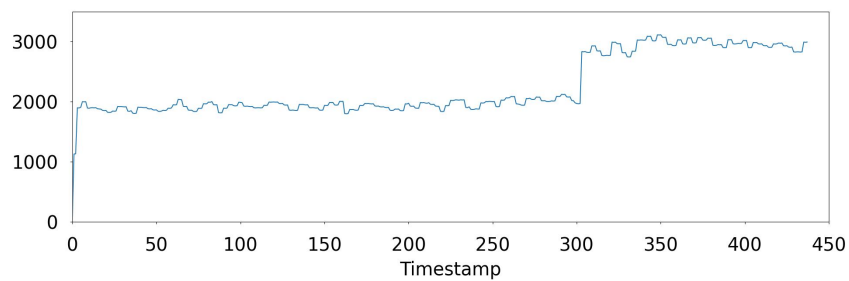
For post-implementation (timing) simulation, we can run the following commands instead

```
1 # file name : launch_simulation.tcl (Post-Implementation, Timing)
2 open_project ProjectVault_AES_Impl.xpr
3 open_run impl_1
4 launch_simulation -mode post-implementation -type TIMING
5 close_sim
6 exit
```

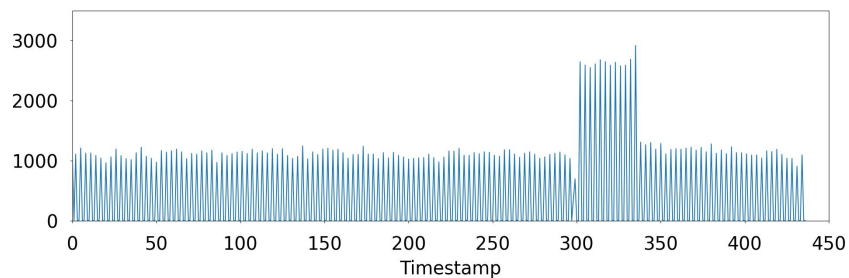
If it is functional, we can replace `TIMING` with `FUNCTIONAL` for `-type`.

We provide example of simulated traces of ProjectVault from the netlist level and gate level in Fig. 29, Fig. 30, Fig. 31 and Fig. 32.

We can also make the changes accordingly in the all-in-one script as well.

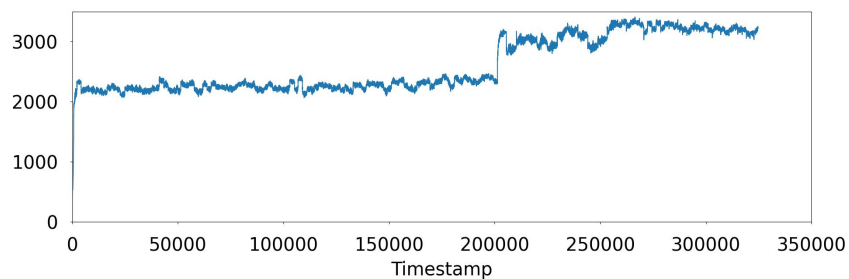


(a) Simulated Trace (Hamming Weight)

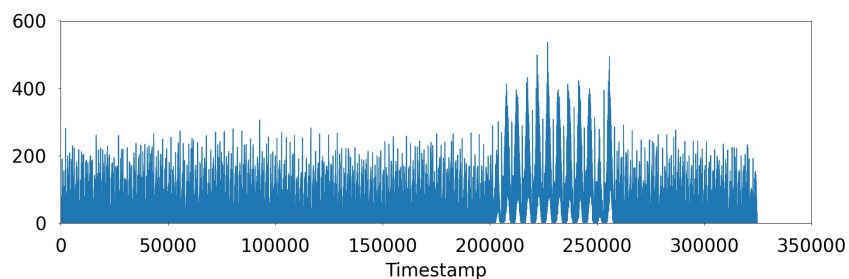


(b) Simulated Trace (Hamming Distance)

Fig. 29: Post Synthesis Functional Simulated trace of ProjectVault running with 2b7e151628aed2a6abf7158809cf4f3c as the key and 6bc1bee22e409f96e93d7e117393172a as the plaintext.

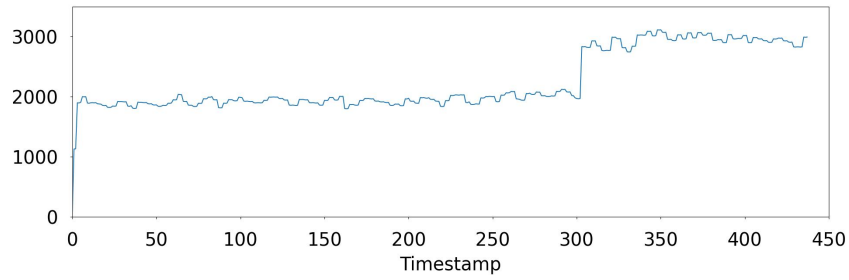


(a) Simulated Trace (Hamming Weight)

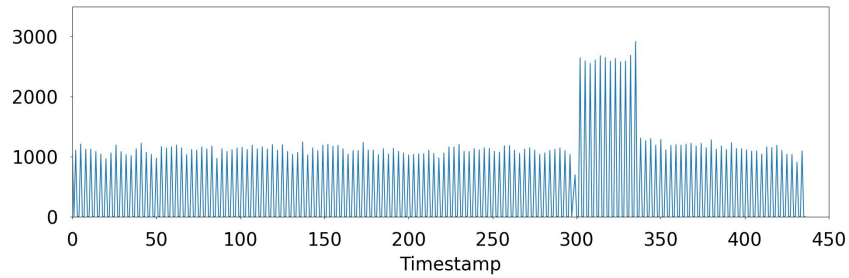


(b) Simulated Trace (Hamming Distance)

Fig. 30: Post Synthesis Timing Simulated trace of ProjectVault running with 2b7e151628aed2a6abf7158809cf4f3c as the key and 6bc1bee22e409f96e93d7e117393172a as the plaintext.

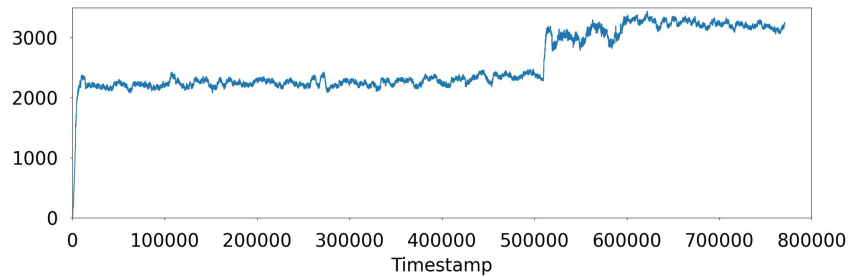


(a) Simulated Trace (Hamming Weight)

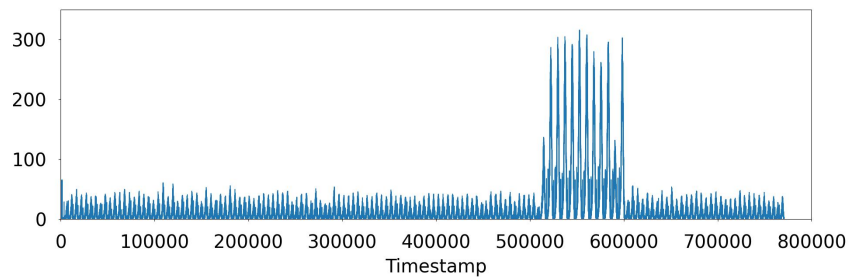


(b) Simulated Trace (Hamming Distance)

Fig. 31: Post Implementation Functional Simulated trace of ProjectVault running with 2b7e151628aed2a6abf7158809cf4f3c as the key and 6bc1bee22e409f96e93d7e117393172a as the plaintext.



(a) Simulated Trace (Hamming Weight)



(b) Simulated Trace (Hamming Distance)

Fig. 32: Post Implementation Timing Simulated trace of ProjectVault running with 2b7e151628aed2a6abf7158809cf4f3c as the key and 6bc1bee22e409f96e93d7e117393172a as the plaintext.

4.13 Bitstream of ProjectVault for ChipWhisperer CW305 FPGA

In addition to the simulated traces from simulation (pre-silicon), it is also useful to being able to capture power traces from a physical device (post-silicon), such as FPGA, to compare the traces and results between the pre-silicon and post-silicon stages. Fortunately, we are able to implement ProjectVault on ChipWhisperer CW305 FPGA board with some additional changes in the our setup. We were not able to achieve this with the unmasked AES design SecWork used in the previous version of our tutorial.

The modified version of ProjectVault and setup we use above is sufficient enough to perform simulation at the RTL, Post-Synthesis, and Post-Implementation in Vivado. In order to run the design on ChipWhisperer CW305 FPGA board and interact with ChipWhisperer Lite for the purpose of trace collection, we include an additional set of wrapper files (including `cdc_pulse.v`, `clocks.v`, `cw305_aes_defines.v`, `cw305_reg_aes.v`, `cw305_main.xdc`, `cw305_top.v`, and `cw305_usb_reg_fe.v`) from ChipWhisperer repository¹¹¹²¹³, and we choose `cw305_top.v` as the top module. In addition, we need to make the following two modifications.

Modification 1. It is worth mentioning that the default configuration of ChipWhisperer’s wrapper files were crafted and suitable for ProjectVault, however, the instance in charge of calling ProjectVault is disabled. To enable it, we add ‘`define GOOGLE_VAULT_AES`’ to the beginning of `cw305_top.v`.

Modification 2. The constraintaintaint file `cw305_top.v` is used to specify port placements, delay, speed, etc. Although everything can be left by default per ChipWhisperer’s guidelines, one should review line 76 to line 78 of `cw305_top.v`, which are used to configure the three clocks of the design: `usb_clk`, `tio_clk`, `p11_clk1`. An example of these lines is presented below as a reference.

```

1 create_clock -period 10.000 -name usb_clk -waveform {0.000 5.000} [get_nets
   usb_clk]
2 create_clock -period 10.000 -name tio_clk -waveform {0.000 5.000} [get_nets
   tio_clk]
3 create_clock -period 10.000 -name p11_clk1 -waveform {0.000 5.000} [get_nets
   p11_clk1]
```

Depending on the design, clock period (specified by `-period`) and half clock period (specified by `-waveform`) should be adjusted such that no timing constraint exists inside the design when Vivado performing scheduling during synthesis stage. A violated timing constraint can lead to a wrongfully synthesized design (producing incorrect input or even not working at all on physical FPGA board). Even if the synthesized design works correctly on the physical FPGA board, operation of that design may damage internal component of the board (not likely but safe than sorry). Those are avoidable risks which should be addressed before attempting to run synthesis and generate the bitstream.

¹¹ <https://github.com/newaetech/chipwhisperer/tree/develop/firmware/fpgas/aes/hdl>

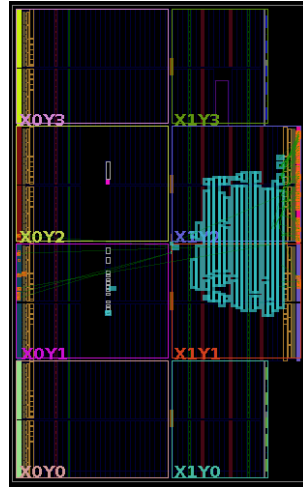
¹² <https://github.com/newaetech/fpga-common/tree/main/hdl>

¹³ <https://github.com/newaetech/chipwhisperer/tree/develop/firmware/fpgas/aes/vivado>

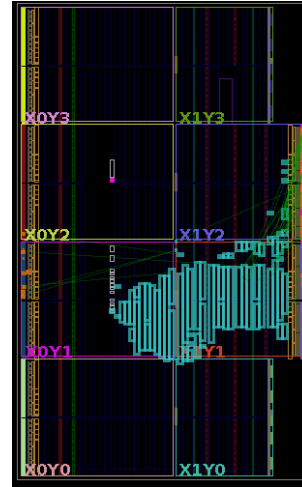
After making the modifications above, we can run “Synthesis”, “Implementation”, and “Generate Bitstream” to obtain a bitstream file. The utilization and layout of this design on CW305 board are reported in Vivado and are presented in Table 1 and Fig. 33.

Table 1: Utilization of AES designs on CW305 FPGA board (part: xc7a100tftg256-2)

	ProjectVault	SMAesH
Power (W)	0.12	0.11
Max Frequency (MHz)	100.58	87.31
Cycles	106	1152
LUTs	4,620 (7.29%)	4,394 (6.93%)
Flip-Flops	2,060 (1.62%)	2,900 (2.29%)
BRAM Blocks	0 (0%)	0 (0%)
DSP Slices	0 (0%)	0 (0%)



(a) ProjectVault (unmasked)



(b) SMAesH (masked)

Fig. 33: Layouts of AES designs on CW305 FPGA board (part: xc7a100tftg256-2)

4.14 Capture Power Traces of ProjectVault on CW305 FPGA

To capture power traces of ProjectVault on CW305 board, we first connect CW305 board with ChipWhisperer Lite as shown in Fig. 34.

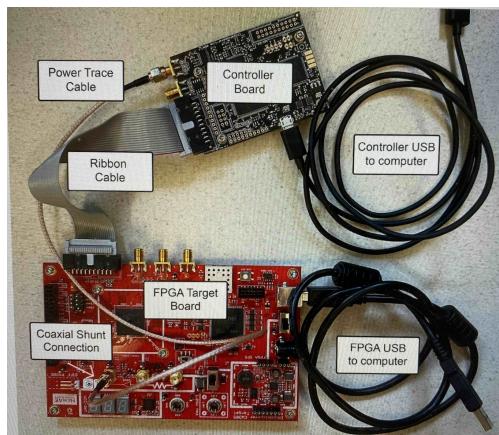


Fig. 34: CW305 FPGA board connected to ChipWhisperer Lite and host machine

Specifically, once the board is connected, we open `capture_normal_data_FPGA_custom.ipynb`¹⁴ and run the first cell to configure ChipWhisperer Lite board with settings related to Analog-to-Digital converter (ADC) and trigger mode. They will be used to make sure 1) only design's operation is captured, 2) trace alignment, and 3) sample analog power consumption signal into its digital version, which will be transmitted back to the host machine.

```

1 # first cell
2 import chipwhisperer as cw
3 scope = cw.scope()
4 scope.gain.db = 25
5 scope.adc.samples = 200
6 scope.adc.offset = 0
7 scope.adc.basic_mode = "rising_edge"
8 scope.trigger.triggers = "tio4"
9 scope.io.tio1 = "serial_rx"
10 scope.io.tio2 = "serial_tx"
11 scope.io.hs2 = "disabled"

```

Next, we run the second cell to program CW305 FPGA board with the bitstream file we generated from Vivado in the previous step.

```

1 # code block from notebook to show the bitstream is uploading to the target.
2 ProjectVault_bsfile = "/home/mdphuc/UCDASEC/SCA/AES_Implementation/AESHardware/
   CW305_ProjectVault/CW305_ProjectVault/CW305_ProjectVault/CW305_ProjectVault.
   runs/impl_1/cw305_top.bit"
3
4 bsfile = ProjectVault_bsfile
5
6 target = cw.target(scope, cw.targets.CW305, fpga_id=FPGA_ID, bsfile=bsfile, force=
   True)

```

¹⁴ https://github.com/UCdasec/HardSide/blob/main/Data_Collection/capture_normal_data_FPGA_custom.ipynb

Once we load the bitstream file successfully, we can run the following code block to capture one trace of the AES design by providing a plaintext and a key.

```

1 # code block from notebook to show the capture of one trace.
2 import numpy as np
3 import time
4 from Crypto.Cipher import AES
5 from tqdm import trange
6 import os
7
8 previous_dataset = None
9 N = 1 # Number of traces
10
11 def endian_reverse(array):
12     return array[::-1]
13
14 if previous_dataset != None:
15     dataset = np.load(previous_dataset)
16     power_trace = list(dataset["power_trace"])
17     plain_text = list(dataset["plain_text"])
18     key = dataset["key"]
19     plaint_text_hex_str = []
20     for plaintext in plain_text:
21         plaint_text_hex_str.append(text_out_to_hex_string(plaintext))
22     print("Current dataset size:", len(power_trace))
23 else:
24     power_trace, plain_text, text_out = [], [], []
25     plaint_text_hex_str = []
26
27 ##### K1 #####
28 key_str = "2b,7e,15,16,28,ae,d2,a6,ab,f7,15,88,09,cf,4f,3c"
29 key = [0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae, 0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0
        x09, 0xcf, 0x4f, 0x3c]
30 text = [0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40, 0x9f, 0x96, 0xe9, 0x3d, 0x7e, 0x11, 0
        x73, 0x93, 0x17, 0x2a]
31
32 ##### K2 #####
33 # key_str = "d2,d5,01,68,82,83,91,43,96,9e,e9,a2,53,a7,52,e1"
34 # key_str = ",".join(key_str.split(",")[:-1])
35 # key = [0xd2, 0xd5, 0x01, 0x68, 0x82, 0x83, 0x91, 0x43, 0x96, 0x9e, 0xe9, 0xa2, 0
        x53, 0xa7, 0x52, 0xe1][::-1]
36 # text = [0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40, 0x9f, 0x96, 0xe9, 0x3d, 0x7e, 0x11,
        0x73, 0x93, 0x17, 0x2a]
37
38 ktp.setInitialKey(key_str)
39 ktp.fixed_key = True

```

44

```

52         f.write("Running_with_plaintext_{0}".format(i + len(dataset["
53             plain_text"])))
54     else:
55         f.write("Running_with_plaintext_{0}".format(i))
56
57     while text_out_to_hex_string(text) in plaint_text_hex_str:
58         key, text = ktp.next() # manual creation of a key, text pair can be
59                                # substituted here
60
61     # FOR ProjectVault
62     trace = cw.capture_trace(scope, target, text, key)
63     # FOR SMAesH
64     # trace = cw.capture_trace(scope, target, bytes(text[::-1]), bytes(key))
65     # FOR CNN
66     # trace = cw.capture_trace(scope, target, bytes(0), bytes(image))
67
68     power_trace.append(np.array(trace[0], dtype=np.float32))
69     plain_text.append(np.array(trace[1]))
70     plaint_text_hex_str.append(text_out_to_hex_string(trace[1]))
71
72     time.sleep(0.15)
73
74     ## send trace for visualization every 100th trace
75     if i % 100 == 0:
76         plot.send(trace)
77         time.sleep(0.15)
78
79 key = np.array(key) ## as key in all iterations are the same
80 power_trace = np.array(power_trace, dtype=np.float32)
81 plain_text = np.array(plain_text)
82
83 print(f"Length_power_trace={len(power_trace)}")
84
85 print(f'Key: {text_out_to_hex_string(key)}')
86 print(f'Text: {text_out_to_hex_string(text)}')
87
88
89 # used to save the data
90 import os
91 output_path = '/home/phucmai/'
92 os.makedirs(output_path, exist_ok=True)
93 outpath = os.path.join(output_path,
94     'CW305_SMAesH_96MHz_test_without_endian_reverse_100k_K2.npz')
95 np.savez(outpath, power_trace=power_trace, plain_text=plain_text, key=key)

```

In order to properly obtain a power trace, we have two recommendations in the notebook. First, we need to adjust the total number of samples per trace (`scope.adc.samples`) to make sure that it is greater than the number of cycles required for ProjectVault to complete. During our experiment, we noticed that ProjectVault took 106 cycles to complete, indicating `scope.adc.samples = 200` should be sufficient for the trace to be captured entirely.

```
1 # Code to adjust scope.adc.samples - this line can be found on line 5 in first
   cell of the jupyter notebook
2 scope.adc.samples = 200
```

Second, the following two blocks should be run (and re-run if needed) multiple times to ensure sampling rate has been correctly configured (it should be around 96MHz).

```
1 ### Running these cells in the following order multiple time to make sure smapling
   rate has been correctly figured
2 # Cell 3
3 target.vccint_set(1.0)
4 # we only need PLL1:
5 target.pll.pll_enable_set(True)
6 target.pll.pll_outenable_set(False, 0)
7 target.pll.pll_outenable_set(True, 1)
8 target.pll.pll_outenable_set(False, 2)
9
10 # run at 10 MHz:
11 target.pll.pll_outfreq_set(10E6, 1)
12
13 # 1ms is plenty of idling time
14 target.clkusbautooff = True
15 target.clksleeptime = 1
16
17 # Cell 4
18 import time
19 for i in range(10):
20     scope.clock.reset_adc()
21     time.sleep(1)
22     if scope.clock.adc_locked:
23         break
24 assert (scope.clock.adc_locked), "ADC failed to lock"
25
26 # Cell 5
27 if scope._is_husky:
28     scope.clock.clkgen_freq = 40e6
29     scope.clock.clkgen_src = 'extclk'
30     scope.clock.adc_mul = 4
31     # if the target PLL frequency is changed, the above must also be changed
       accordingly
32 else:
```

```

33 scope.clock.clkgen_freq = 96e6 / 4
34 scope.clock.clkgen_src = "extclk"
35 scope.clock.adc_src = "clkgen_x4"
36
37 # scope.clock.clkgen_freq = 96e6

```

During our experiment, we noticed that there were some scenarios where the sampling rate stays at 14MHz after running following two blocks the first time. Running those blocks again should resolve the problem and get the sampling rate around 96MHz. We can verify the sampling rate later in this notebook by running the block below:

```

1 print("Capture_rate:␣"+str(scope.clock.adc_freq/10e5)+"MHz")

```

We plot one trace that we capture from the CS305 board in Fig. 35 by running the script below:

```

1 # script to generate the plot.
2 import matplotlib.pyplot as plt
3 import numpy as np
4 plt.rcParams.update({'font.size': 16})
5 plt.figure(figsize=(10,5))
6 plt.clf()
7 plt.xlabel("Timestamp")
8 plt.ylabel("Normalized_Voltage_Drop")
9 plt.plot(power_trace[0])

```

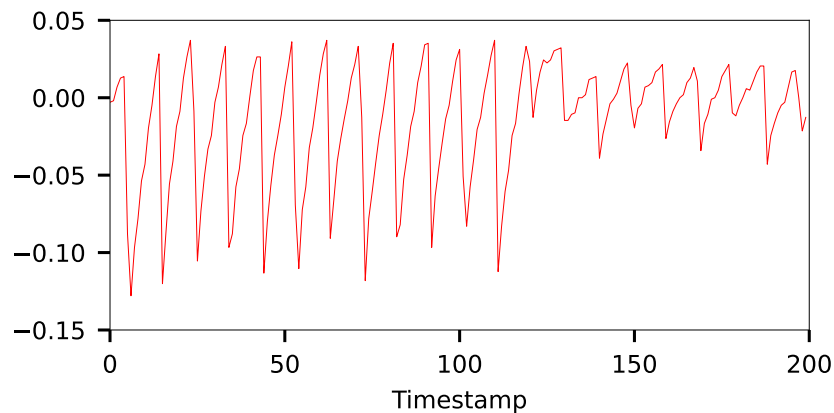


Fig. 35: A power trace of ProjectVault running on CW305 (sampling rate: 96 Mhz)

Operations behind the Scene. To help readers better understand the process of obtaining power traces on CW305 board, we also explain additional details regarding how ChipWhisperer interacts with the bitstream on board as shown in Fig 36. First, ChipWhisperer Lite is configured with **1)** the number of samples needed to be record, **2)** pattern to trigger, **3)** clock frequency for

the target board, etc. Once everything is in place, ChipWhisperer Lite will program the CW305 FPGA board with ProjectVault's bitstream.

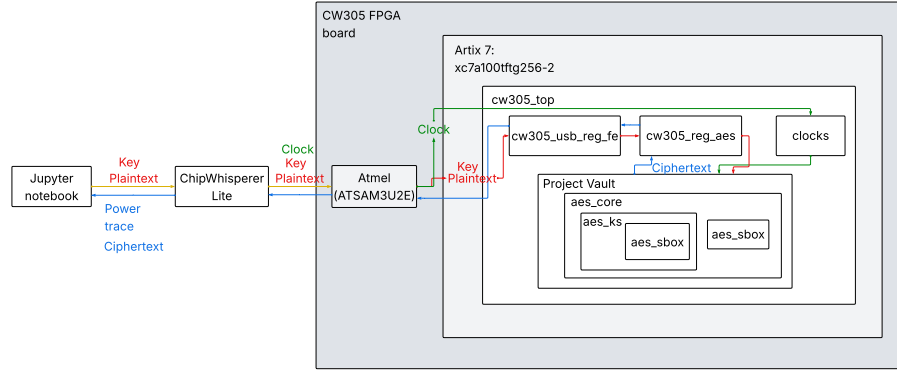


Fig. 36: Operations behind the scene, ProjectVault running on CW305 FPGA board interacting with ChipWhisperer Lite

Next, to capture one power trace, **1)** Jupyter notebook sends user-defined key and plaintext to CW Lite, **2)** CW Lite generates clock signal for CW305 FPGA board based on the value predefined by user and sends both key, plaintext, and the clock signal to CW305, specifically to the Atmel (ATSAM3U) microcontroller unit via USB interface, **3)** Atmel passes the clock signal, key and plaintext respectively to ProjectVault's `clocks` and `cw305_usb_reg_fe` module, **4)** key and plaintext then travel to `cw305_reg_aes` to `aes_core`, while clock signal travels from `clocks` to `aes_core`, **5)** AES operation begins and CW Lite starts recording one power trace and **6)** once the operation has concluded, ciphertext is pulled via the same path taken by key and plaintext to the Jupyter notebook itself; stops recording the power trace and sends the obtained result back to the Jupyter notebook. It normally takes around 0.25 seconds for capturing all the samples in 1 trace. The time increases linearly with the number of traces we like to collect. For instance, to collect 200,000 power traces, it takes about 14 hours.

This concludes the tutorial on unmasked AES with ProjectVault.

5 Generating Simulated Traces for Masked AES

In this section, we describe the details steps for how to generate traces at the RTL level by passing a key and plaintexts to another masked AES hardware implementation named SMAesH. The flow of the entire pipeline is almost the same as the one presented previously for ProjectVault. While we try to focus on the differences, we aim to make the presentation as self-contained as possible.

SMAesH is a masked hardware implementation of the Advanced Encryption Standard (AES) developed by SIMPLE-Crypto¹⁵. There was also an evaluation challenge on SMAesH using post-silicon side-channel analysis from CHES2023¹⁶. In general, SMAesH uses Hardware Private Circuit (HPC), HPC1 and HPC3, masking standard as a defense mechanism against side-channel attack, specifically probing attack [5,9]. There are multiple versions provided. We utilize the version v1.1.0 as shown in Fig. 37

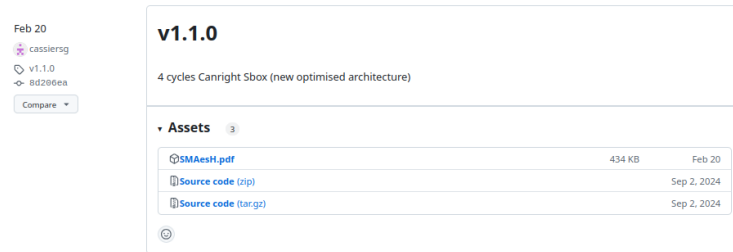


Fig. 37: SMAesH (version 1.1.0) Github Download Page

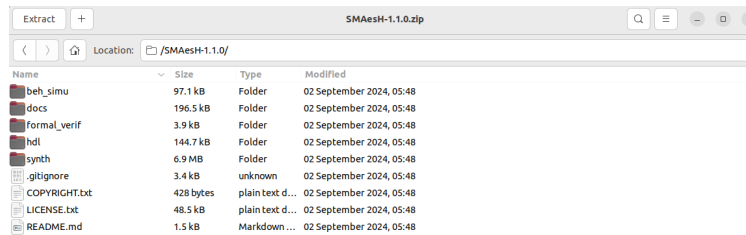


Fig. 38: Overview of SMAesH v1.1.0

5.1 Download the Source Code of SMAesH

1. Download the ZIP file from the Github repository of SMAesH at <https://github.com/simple-crypto/SMAesH/releases/tag/v1.1.0>

¹⁵ <https://www.simple-crypto.org/activities/smaesh/>

¹⁶ <https://smaesh-challenge.simple-crypto.org/>

2. Extract the the ZIP file `SMAesH-1.1.0.zip` to your home directory. This creates a directory named `./SMAesH-1.1.0`, whose structure can be found in 38. The design of SMAesH locates under `./hdl/aes_enc128_32bits_hpc` and `./hdl/nshares_<# of shares: 2, 3, or 4>`. The testbench file (`tb_aes_enc128_32bits_hpc.v`) can be found in `./hdl/tb`. By running `make simu_build` inside `./beh_simu`, all SMAesh's design files are copied to `./beh_simu/work/nshares_<# of shares: 2, 3, or 4>/hdl` (by default, `nshares` is set to 2).

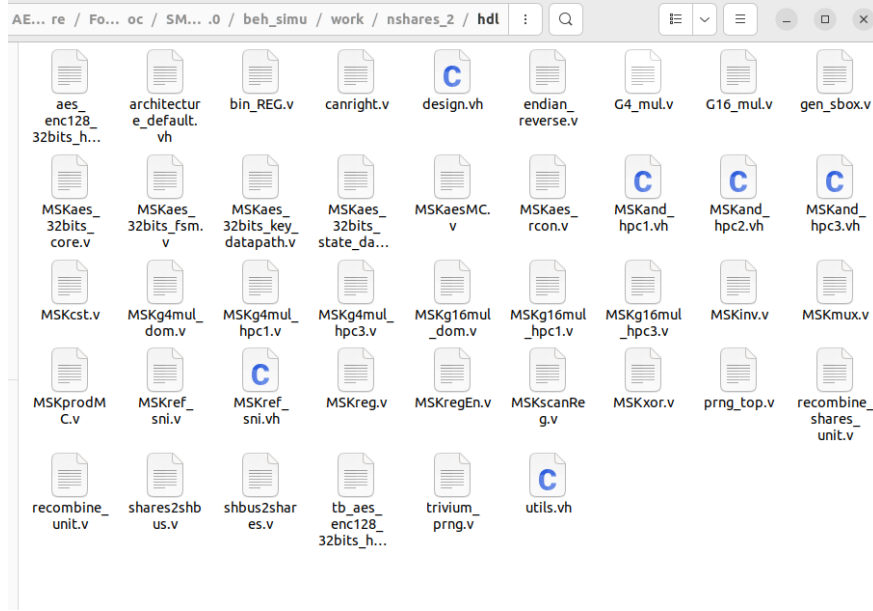


Fig. 39: SMAesH's design sources for `nshares = 2`

5.2 Overview

In this section, we describe the detailed steps for establishing our siulation pipeline and how to generate simulated traces at the RTL level by passing a key and plaintexts to the AES hardware implementation we use (i.e. SMAesH).

Overview. We will first create a Vivado project based on the source code of SMAesH. Once the project is created, we will need to modify the original code properly to make it compatible with downstream parsing tool (TOFU), and run RTL simulation to generate a VCD file. Once a VCD file is obtained, we will install and run TOFU to generate a simulated trace. We describe details of each step below.

5.3 Create a Project in Vivado for AES Implementation

We need to create a new project for the Masked AES hardware implementation (SMAesH) by following the steps below.

1. Launch Vivado 2023.2 and click on “Create Project” (same as unmasked AES).

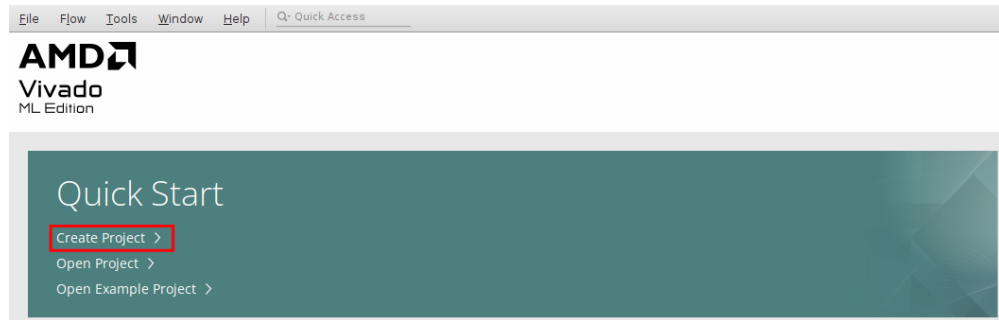


Fig. 40: Step 1: Create a new project in Vivado

2. Follow the “Create New Project” wizard, create a project name (SMAesH in our example), set the location to home directory: `/home/username` (`username` is `mdphuc` in this running example), check the box “Create project subdirectory”, and click “Next”

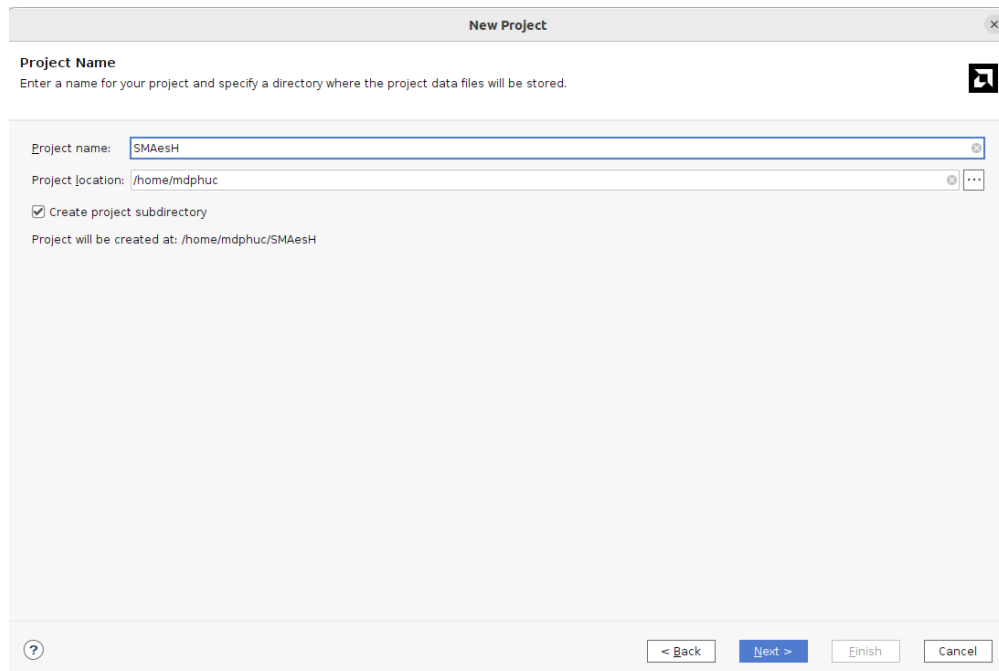


Fig. 41: Step 2: Create project name and choose project location (SMAesH)

3. Specify the project type as an RTL project, and click “Next” (same as unmasked AES)

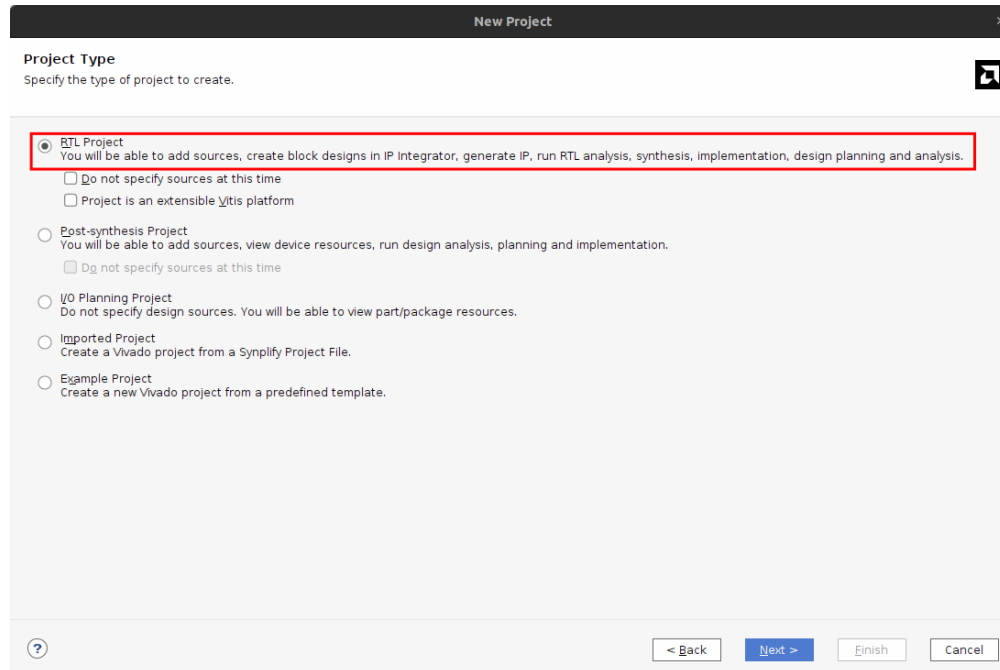


Fig. 42: Step 3: Choose project type as an RTL project (SMAesH)

4. On the “Add Sources” screen, click “Add Files” and add the 41 source files, as shown in 43 (except for `tb_aes_enc128_32bits_hpc.v`) from `./beh_simu/work/nshares_2/hd1` directory to the project, then click “OK” to return to the “Add Sources” screen.

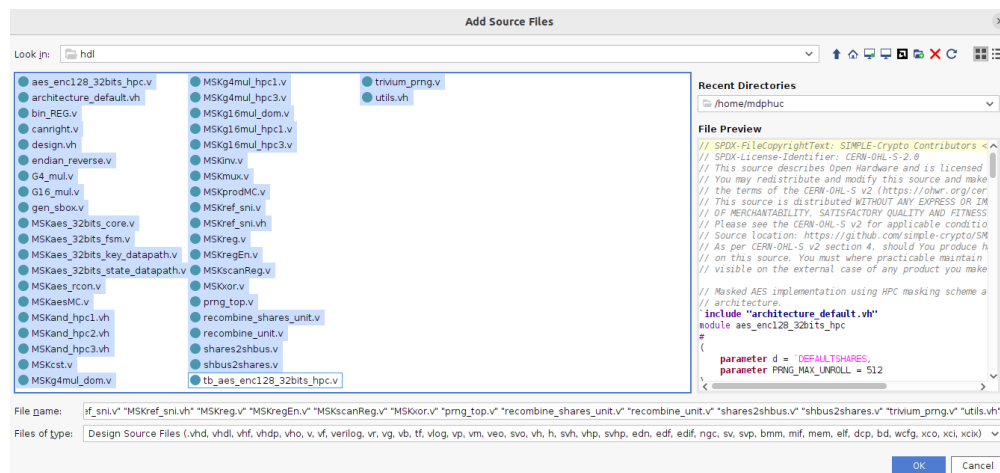


Fig. 43: Step 4: Add source files of SMAesH to Vivado project (SMAesH)

5. Click “Next” to proceed to add the files and the testbench file to our project. You will see all the added files listed.

6. Set the Default AMD part as: `xc7a100tftg256-2` (we use ChipWhisperer CW305 FPGA board). We choose this one as we have access to CW305 FPGA board in our lab. Other parts can also be selected. Based on our experience, two different parts will lead to the same simulated trace at the RTL level (as the RTL code remains the same) but their simulated traces will have discrepancies at the netlist level and gate level.

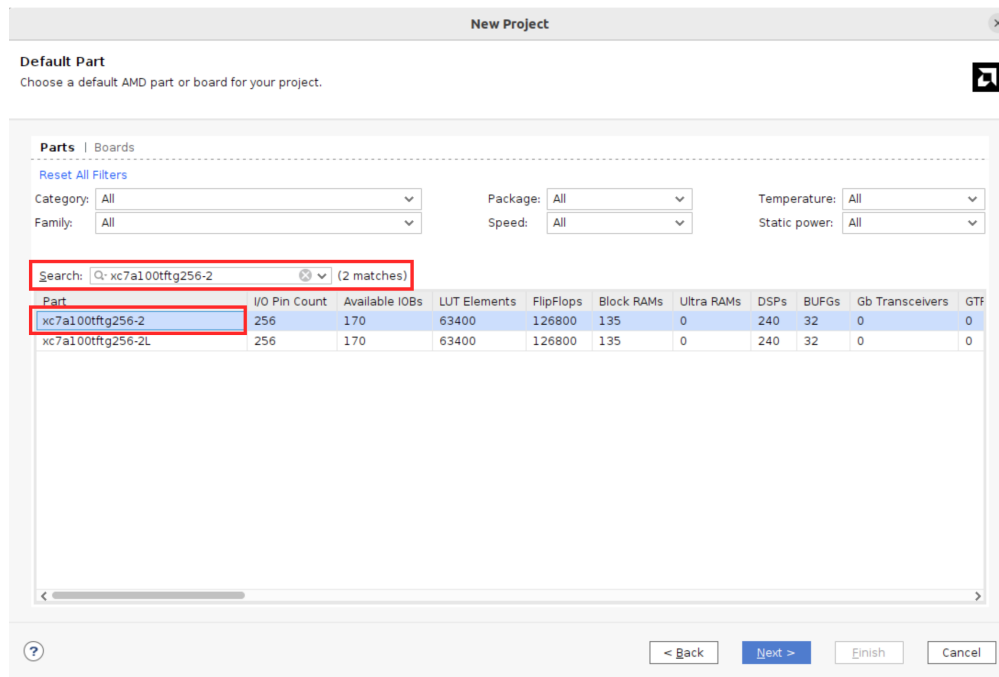
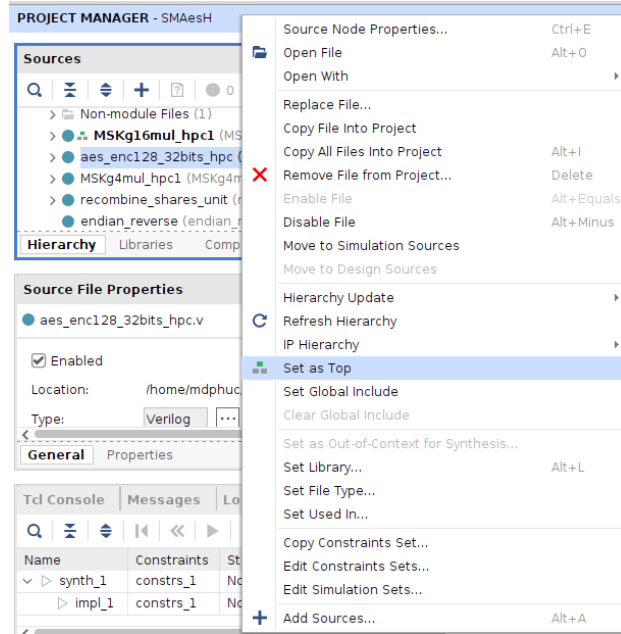


Fig. 44: Step 7: choose the default part for the project

7. After loading into the project and waiting for the project to update, by default, Vivado sets `MSKg16mul_hpc1.v` as the top module in the project, despite `aes_enc128_32bits_hpc.v` being the correct one. To address this, we may need to right-click on `aes_enc128_32bits_hpc.v` and select “Set as Top”. This completes the initial setup for the project in Vivado. *It is worth mentioning that we will need to slightly adjust the top module later on for this implementation.* We will discuss the details in Sub-section 5.4.

Fig. 45: Step 6: Set `aes_enc128_32bits_hpc.v` as the top module

5.4 Modify SMAesH.

Examining the source codes, we noticed `aes_enc128_32bits_hpc.v` is the main module of the design; however, in order for the design to work, the testbench program also has to incorporate some other modules (i.e. `endian_reverse` and `recombine_shares_unit`). The design also uses some synthesis flags that are not supported by Vivado (i.e. line 20 of `MSKaes_32bits_core.v`). It is also worth mentioning that the calculated ciphertext is presented in reverse order. With that in mind, we make the following modifications to SMAesH, including (I) removing any flags that are not compatible with Vivado, (II) creating a new top module that incorporates main module `aes_enc128_32bits_hpc` and any supporting modules and callings, and (III) reversing the calculated ciphertext before pushing it to the output stream.

Modification I. We need to remove or comment out the following lines.

```

1 line 20, MSKaes_32bits_core.v
2 line 15, MSKaes_32bits_state_datapath.v
3 line 17, MSKprodMC.v
4 line 17, MSKaesMC.v
5 line 15, MSKaes_32bits_key_datapath.v

```

Specifically, we noticed that on these lines, SMAesH utilizes a synthesis flag named `fv_order` and assigns `d` to it, which is flagged by Vivado during Synthesis. By examining the code, we realized that `d` here is likely to be parameter `d` belonged to the main module (i.e. `MSKaes_32bits_core`). It seems that Vivado cannot associate the two `d`, so an error was flagged at that location during synthesis. Examining further, we found that those synthesis flags (i.e. `fv_prop`, `fv_strat` and `fv_`

`order` in `MSKaes_32bits_core.v`) are not supported by Vivado per their documentation as shown in ¹⁷. As we mainly use Vivado for the purpose of pre-silicon side channel analysis, it is safe to remove those synthesis flags from the design as they do not contribute to the final netlist implementation of the design.

Modification II. The original implementation obtains a ciphertext, but the bytes are in the reverse order. We initially planned on integrating a module to reverse the bytes to the correct order; however, when running Pre-Silicon Side-channel Analysis, this module unexpectedly introduced unwanted leakage to the design. Due to this, we leave SMAesH as it is and reformat the plaintext and key into little endian order in advance before running SMAesH.

Modification III. We notice that the use of two `shares2shbus` within top module `aes-enc128_32bits_hpc.v` causes the synthesized design to behave incorrectly. Because of this, we update the `shares2shbus` module to accept both plaintext and key at the same time. It's worth mentioning that as the original module `shares2shbus` is also used by other modules in the design (i.e. `recombine_shares_unit.v`, etc.), removing such module might cause catastrophic effect to the entire design. Because of that, we decided to add another module named `shares2shbus2` as a solution to this problem. An example of such module can be found as below:

```

1 // shares2shbus2.v
2
3 module shares2shbus2
4 #
5 (
6     parameter d = 2,
7     parameter count = 128
8 )
9 (
10    shares_key,
11    shbus_key,
12    shares_plaintext,
13    shbus_plaintext
14 );
15
16 // IOs
17 input [255:0] shares_key;
18 output [255:0] shbus_key;
19
20 input [255:0] shares_plaintext;
21 output [255:0] shbus_plaintext;
22
23 genvar i,j;
24 generate
25 for(i=0;i<count;i=i+1) begin: bit_wiring_key
26     for(j=0;j<d;j=j+1) begin: share_wiring_key

```

¹⁷ <https://docs.amd.com/r/en-US/ug912-vivado-properties>

```

27         assign shbus_key[d*i + j] = shares_key[count*j + i];
28     end
29 end
30 endgenerate
31
32 genvar k,m;
33 generate
34 for(k=0;k<count;k=k+1) begin: bit_wiring_plaintext
35     for(m=0;m<d;m=m+1) begin: share_wiring_plaintext
36         assign shbus_plaintext[d*k + m] = shares_plaintext[count*m + k];
37     end
38 end
39 endgenerate
40
41
42 endmodule

```

Modification IV. We also notice that the output ciphertext is represented in shares format instead of hexadecimal. As a result, we also add another instance of `recombine_shares_unit.v` to top module `aes_enc128_32bits_hpc.v` to convert ciphertext back to its hexadecimal form. An example of such update can be found as below, where output ciphertext (`out_shares_ciphertext`) is compiled into hexadecimal form and stored in variable `aes_msk_cipher`.

```

1 // aes_enc128_32bits_hpc.v
2
3 recombine_shares_unit #(d('D'),.count(128))
4 ru(
5     .shares_in(out_shares_ciphertext),
6     .out(aes_msk_cipher)
7 );

```

As mentioned above in Sub-section 5.3, we need to adjust the top module of the design. Specifically, we make `aes_top.v` as top module of the design by right-clicking on `aes_top.v` and selecting “Set as Top”. This concludes all modifications needed for SMAesH.

5.5 Simulate the AES Hardware Design in Vivado.

With all the modifications we mentioned in the previous subsection, we are ready to simulate the hardware design of Masked AES-128 given a key and a plaintext and produce a VCD file using Vivado. The process is the same as the one we have for the simulation of unmasked AES. The testbench program we used in this example can be found in Appendix.

1. In Vivado, go to “Simulation” in the “Flow Navigator” pane and click “Run Simulation” then “Run Behavioral Simulation” (same as unmasked AES).

- The terminal in Vivado shows that the RTL simulation of the hardware AES-128 implementation lasts for **442 nanoseconds in simulation** and the encryption runs correctly (we checked in advance). The entire simulation takes some additional time to save the VCD file. Overall, it takes about **13 seconds** to complete the simulation and save one VCD file for this example. Screenshots of the simulation can be found in Fig. 46 and the simulation log is also presented below as a reference.

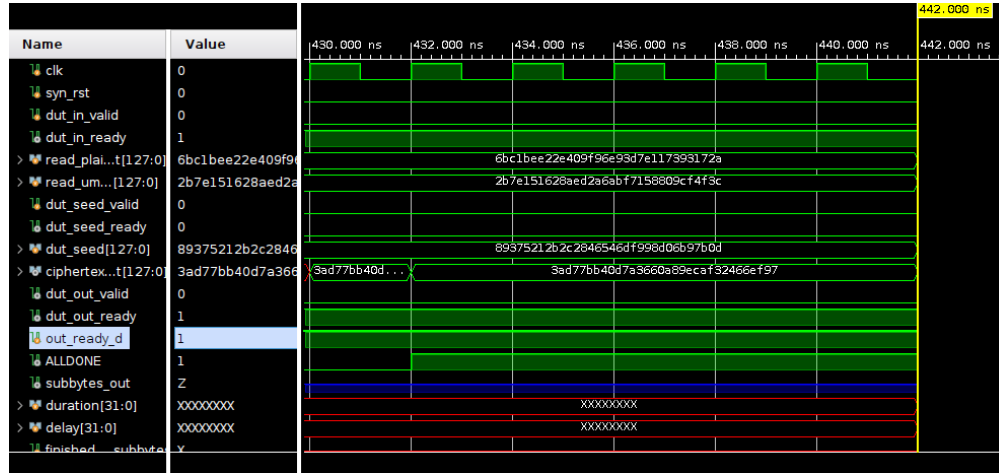


Fig. 46: One simulation given a key and a plaintext with SMAesH

```

1 # simulation log of SMAesH
2 Reseed of PRNG requested...
3 New seed of PRNG fetched!
4 Ciphertext = 3ad77bb40d7a3660a89ecaf32466ef97
5 $finish called at time : 442 ns : File "/home/mdphuc/UCDASEC/SCA/
   AES_Implementation/AESHardware/For_TOFU_Overleaf_doc/SMAesH-1.1.0/hdl/tb/
   SMAesH\_tb.v" Line 177
6 INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_behav' loaded.
7 INFO: [USF-XSim-97] XSim simulation ran for 1000ns
8 launch_simulation: Time (s): cpu = 00:00:24 ; elapsed = 00:00:13 . Memory (MB):
   peak = 8303.426 ; gain = 105.992 ; free physical = 673 ; free virtual =
   3344

```

- Once the simulation completes, locate the generated VCD file `trace.vcd` by following the path below (this depends on the name of the project)

```

1 /home/mdphuc/SMAesH/SMAesH.sim/sim_1/behav/xsim/trace.vcd

```

Once we obtain a VCD file, the remaining steps to generate a simulated trace (i.e. removing empty lines in VCD and running TOFU) are the same as the ones in the unmasked AES (except the path of the VCD file). We skip the details here.

5.6 An Example of a Simulated Trace

Assume that we generate a simulated trace in `./tofu_master/temp_masked/trace.h5`, we can generate a plot to visualize it by using the following Python script.

```

1 # file_name: openoneTrace.py (same script before but update file names)
2 import h5py
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 with h5py.File('/home/username/tofu-master/temp_masked/trace.h5', 'r') as file:
7
8     dataset = file['/leakages']
9     # Extracting the data from the dataset
10    data = dataset[:, :]
11
12    # Reshape the data to a 1D array
13    data = data.reshape(-1)
14    plt.plot(data)
15    plt.figure(figsize=(10, 6))
16
17    plt.tight_layout()
18    plt.show()
19    plt.savefig('/home/username/tofu-master/temp_masked/trace.png')

```

One example trace based on Hamming Distance model is presented in Fig. ?? . It consists of 440 samples, where each sample is based on the toggle counts at each nanosecond within each clock cycle from the RTL simulation in Vivado. Another example of a simulated trace based on Hamming Weight model is presented in Fig. ?? . It consists of 441 samples, one more sample compared to the number of samples in a trace produced using Hamming Distance.

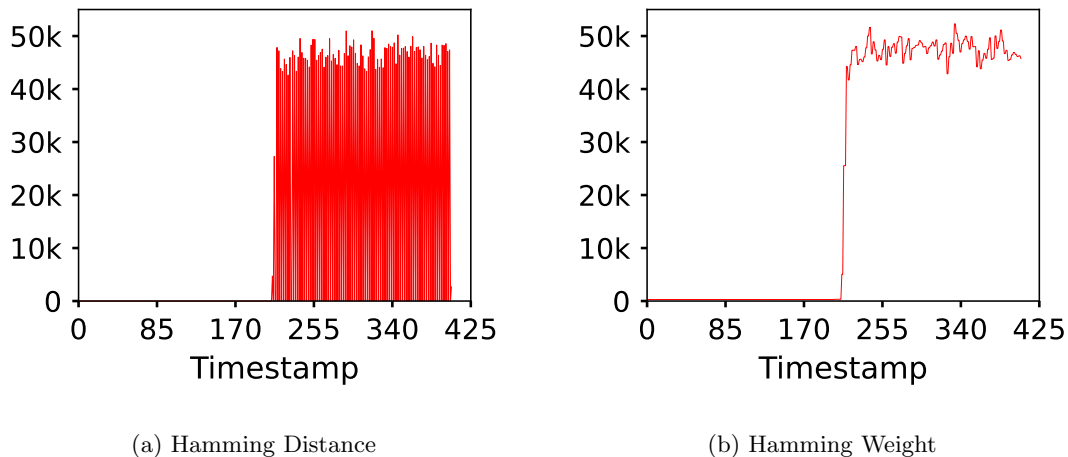


Fig. 47: Simulated traces generated by our pipeline (Vivado + SMAesH + TOFU).

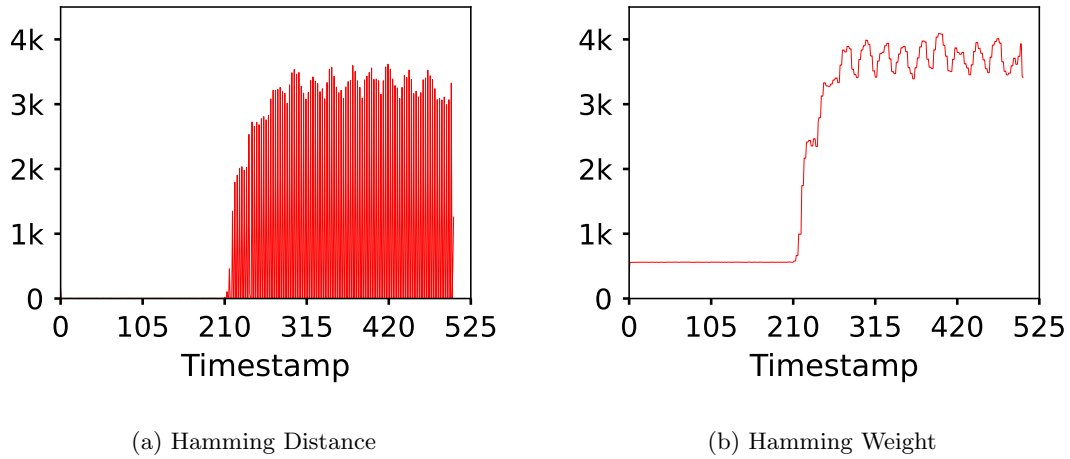


Fig. 48: Simulated traces (Post-Synthesis) generated by our pipeline (Vivado + SMAesH + TOFU).

5.7 Run Our Simulation to Obtain Multiple Simulated Traces

The process of obtaining multiple simulated traces for SMAesH is almost the same as the one described in unmasked AES; however, we need to update the testbench program (`SMAesH_tb.v`) to incorporate the seed supply and update functionality. Specifically, we store the seed in a file named `seed.txt`, then during each run time, `SMAesH_tb.v` reads the seed from `seed.txt`, feeds it to SMAesH, and once valid calculated ciphertext has been received, `SMAesH_tb.v` increments it by 1 and then writes it back to `seed.txt` for the next run. It is worth mentioning that the updates will be done on SMAesH's testbench as shown in Appendix. Our modification process then can be found as below:

1. Follow **Additional Modification in `tb_aes.v`** as highlighted in unmasked AES section 4.9 to update `SMAesH_tb.v`. Specifically, we inject a new task block into SMAesH's testbench (as shown in Appendix) to load additional "config.txt" file, located in the same directory as the base Vivado project. Finally, we add an additional line in the main execution block to call this task, "loadConfig()":

```

1 // New task to load the config file for plaintext and iteration number
2 task loadConfig;
3     begin
4         fileHandler = $fopen("config.txt","r");
5         if (fileHandler) begin
6             $fscanf(fileHandler, "%x,%d", read_plaintext, plaintextNum);
7             $fclose(fileHandler);
8         end
9         $sformat(vcdFilename, "plaintext%0d.vcd", plaintextNum);
10        $dumpfile(vcdFilename);
11        $dumpvars(0, tb_aes);

```

```

12         $display("Dumping to: %s", vcdFilename);
13     end
14 ...
15 ...
16 ...
17 initial begin
18     clk = 1;
19     syn_rst = 1;
20     out_ready_d = 1;
21
22     dut_in_valid = 0;
23
24     // Init delay
25     #(wait_delay);
26     #(init_delay);
27
28     // Reset sequence
29     syn_rst = 1;
30     #T;
31     syn_rst = 0;
32     #T;
33
34     #(init_delay);
35
36     $display("reg_init_reseed_done = %x", reg_init_reseed_done);
37
38     // Wait that init reseed is done
39     wait(reg_init_reseed_done);
40     // Input feeding
41     loadConfig();
42     read_umsk_key = 128'h2b7e151628aed2a6abf7158809cf4f3c;
43
44     dut_in_valid = 1;
45     wait(dut_in_ready);
46     #T;
47     dut_in_valid = 0;
48
49     wait(ALLDONE);
50
51     $display("Ciphertext = %x", ciphertext_out);
52
53     #10; $finish;
54 end
55 endmodule // tb_aes

```

2. Insert the following code block into `task loadConfig`. This code block is used to read input seed from `seed.txt` and store it in `dut_seed`.

```

1 if (seedHandler) begin
2     $fscanf(seedHandler, "%x", dut_seed);
3     $fclose(seedHandler);
4 end

```

3. Insert the following task (saveSeed) into SMAesH.tb.v and insert `saveSeed()` right before `$finish` in the `initial begin` block. This code block is used to update input seed for next run time. The path the follows `$fopen` is arbitrary and depends on where user wants `seed.txt` to locate at.

```

1 task saveSeed;
2     begin
3         seedHandler = $fopen("/home/mdphuc/UCDASEC/AES_Implementation/
4             AESHardware/SMAesH-1.1.0/seed.txt", "w");
5         if (seedHandler) begin
6             $fwrite(seedHandler, "%x,%x", dut_seed + 1, dut_seed_out[79:0]);
7         end
8     end
9 endtask

```

At this point, we have successfully updated SMAesH’s testbench file for the purpose of automatic large-scale trace collection. The final step is to run the automatic trace collection script, `vivado_all_in_one_tofu_capture.py`, (as explained in the **Additional Modification in tb-aes.v** highlighted in unmasked AES section 4.9).

It is worth mentioning that one VCD file from SMAesH takes up around **10 MBs**, indicating **50,000 traces** may take up around **500 GBs**. However, the intermediate files (raw trace + h5s + pickles) will be compressed at the end of the trace collection process to produce a zip file with approximately only **110 GBs**. The final dataset in `.npz` format will only use around **100 MBs** of disk space.

5.8 Bitstream of SMAesH for ChipWhisperer CW305 FPGA

To run SMAesH on the CW305 board, besides all the existing source files used in the simulation, we include the same set of ChipWhisperer wrapper files as in the case of ProjectVault. We set `cw305_top.v` as the top module. Different from ProjectVault, SMAesH is not configured with the CW305 board by default. To address this, we add a finite state machine to `cw305_top.v`, where the logic of this finite state machine is the same as the stimulus block in SMAesH’s testbench program (`tb/tb_aes_enc128_32bits_hpc.v`). We incorporate the trigger signal (`tio_trigger`) to this finite state machine by setting it to 0 before and after the encryption and assigning it to 1 during the encryption. We keep `(* keep_hierarchy = "yes", DONT_TOUCH = "yes" *)` before each module following the guidelines from the SMAesH Challenge in CHES2023. We adjust the clock periods to 100ns in file `cw305_main.xdc`.

After making the modifications above, we can run “Synthesis”, “Implementation”, and “Generate Bitstream” to obtain a bitstream file. The utilization and layout of this design on CW305 board are reported in Vivado and are presented in Table 1 and Fig. 33.

Once we obtain the bitstream of this design, the remaining steps to connect CW305 board and capture power traces of this bitstream on CW305 is the same as the ones described for ProjectVault in Sec. 4.14. We skip the details here. An example of the power trace is presented in Fig. 49. The trace consists of 1,500 samples.

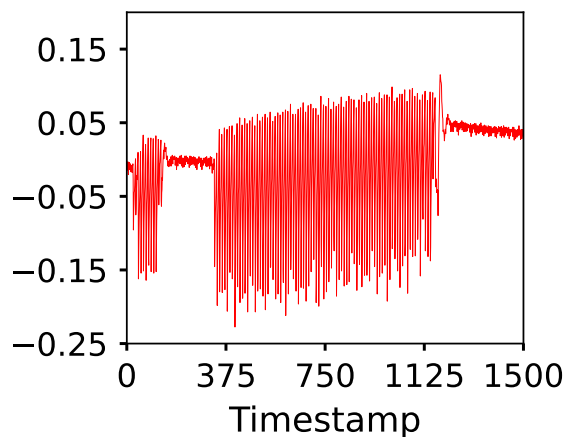


Fig. 49: A power trace of SMAesH running on CW305 (sampling rate: 96 Mhz)

References

1. Hardside.
2. AMD. Vivado - ML Edition. <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html>, 20xx.
3. Gaëtan Cassiers and Charles Momin. The SMAesH dataset, 2024.
4. DaSec. SideRTL. <https://github.com/UCdasec/SideRTL>, 2020.
5. D. Knichel and A. Moradi. Low-Latency Hardware Private Circuits. *CCS '22: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022(1), 2022.
6. ProjectVault. Verilog implementation of AES-128, 2015.
7. Logan Reichling, Phuc Mai, Muhib Khan, Kaiden Thomas, and Boyang Wang. HardSide: Pre-silicon Side-Channel Anlysis on Hardware AES with Deep Learning.
8. tueisec. TOFU - Toggle Count Analysis made simple. <https://gitlab.lrz.de/tueisec/tofu>, 2022.
9. A. Sahai Y. Ishai and D. Wagner. Private circuits: Securing hardware against probing attacks. In *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*, volume 2729 of *Lecture Notes in Computer Science*, pages 463–481. Springer, 2003.

Our Testbench (tb.v) for SMAesH

```

1  `timescale 1ns/1ps
2
3  `ifndef NSHARES
4  `define NSHARES 2
5  `endif
6  (* keep_hierarchy = "yes", DONT_TOUCH = "yes" *)
7  module tb_aes_enc128_32bits_hpc
8  #
9  (
10     parameter T = 4,
11     parameter d = `NSHARES,
12     parameter RND_RANGE_LAT_RESEED=600,
13     parameter RND_RANGE_LAT_IN=100,
14     parameter CONTINUOUS = 0
15 )
16 ();
17
18 localparam wait_delay = 100*T;
19 localparam Td = T/2.0;
20 localparam init_delay=Td/2.0;
21
22 `include "utils.vh"
23
24
25 reg clk;
26 reg syn_rst;
27

```

```

28 reg dut_in_valid;
29 wire dut_in_ready;
30
31 wire [128*d-1:0] dut_shares_plaintext;
32 wire [128*d-1:0] dut_shares_key;
33
34 reg dut_seed_valid;
35 wire dut_seed_ready;
36 reg [127:0] dut_seed;
37
38 wire [128*d-1:0] dut_shares_ciphertext;
39 wire dut_out_valid;
40 wire dut_out_ready;
41
42 reg [127:0] seed_random;
43
44 // Generate the clock
45 always #Td clk=~clk;
46
47
48 // Dut
49 aes_enc128_32bits_hpc
50 `ifdef behavioral
51 #(
52     .d(d)
53 )
54 `endif
55 dut(
56     .clk(clk),
57     .rst(syn_rst),
58     .in_valid(dut_in_valid),
59     .in_ready(dut_in_ready),
60     .in_shares_plaintext(dut_shares_plaintext),
61     .in_shares_key(dut_shares_key),
62     .in_seed_valid(dut_seed_valid),
63     .in_seed_ready(dut_seed_ready),
64     .in_seed(dut_seed[79:0]),
65     .out_shares_ciphertext(dut_shares_ciphertext),
66     .out_valid(dut_out_valid),
67     .out_ready(dut_out_ready)
68 );
69
70 /// Value read from files
71 reg [127:0] read_plaintext;
72 reg [127:0] read_umsk_key;
73 reg [127:0] read_umsk_ciphertext;
74 reg read_last_in;

```

```

75 reg read_last_out;
76
77 // Recombine unit
78 wire [127:0] rec_dut_ciphertext;
79 recombine_shares_unit #(d(d),.count(128))
80 ru(
81     .shares_in(dut_shares_ciphertext),
82     .out(rec_dut_ciphertext)
83 );
84
85 reg [63:0] fileHandler;
86 reg [63:0] seedHandler;
87 reg [25*400:0] vcdFilename;
88
89 reg [31:0] plaintextNum;
90
91 wire [127:0] dut_seed_out;
92
93 assign dut_shares_plaintext[255:128] = 0;
94 assign dut_shares_plaintext[127:0] = read_plaintext;
95 assign dut_shares_key[255:128] = 0;
96 assign dut_shares_key[127:0] = read_umsk_key;
97 //task loadConfig;
98 //     begin
99 //         fileHandler = $fopen("/home/mdphuc/UCDASEC/SCA/AES_Implementation/
100 //             AESHardware/CW305_SMAesH_address_endian_reverse/config.txt","r");
101 //         seedHandler = $fopen("/home/mdphuc/UCDASEC/SCA/AES_Implementation/
102 //             AESHardware/CW305_SMAesH_address_endian_reverse/seed.txt","r");
103 //         if (fileHandler) begin
104 //             $fscanf(fileHandler, "%x,%d", dut_shares_plaintext[127:0],
105 //                 plaintextNum);
106 //             $fclose(fileHandler);
107 //         end
108 //         if (seedHandler) begin
109 //             $fscanf(seedHandler, "%x", seed_random);
110 //             $fclose(seedHandler);
111 //         end
112 //         $sformat(vcdFilename, "/home/mdphuc/UCDASEC/SCA/AES_Implementation/
113 //             AESHardware/CW305_SMAesH_address_endian_reverse/SIMULATED_TRACE/plaintext%0d.
114 //             vcd", plaintextNum);
115 //         $display(vcdFilename);
116 //         $dumpfile(vcdFilename);
117 //         $dumpvars(0, dut);
118 //         $display("Dumping to: %s", vcdFilename);
119 //     end
120 //endtask

```

```

117 //task saveSeed;
118 //    begin
119 //        seedHandler = $fopen("/home/mdphuc/UCDASEC/SCA/AES_Implementation/
    AESHardware/CW305_SMAesH_address_endian_reverse/seed.txt","w");
120 //        if (seedHandler) begin
121 //            $fwrite(seedHandler, "%x,%x", seed_random + 1, dut_seed[79:0]);
122 //        end
123 //    end
124 //endtask
125
126 /////// RUN
127 integer RUN_AM;
128
129 integer id_tv_in;
130 integer id_tv_out;
131
132 // Reseeding mechanism
133 reg reg_init_reseed_done;
134 reg set_init_reseed_done;
135 always@(posedge clk)
136 if(syn_rst) begin
137     reg_init_reseed_done <= 0;
138 end else begin
139     reg_init_reseed_done <= reg_init_reseed_done | set_init_reseed_done;
140 end
141
142 integer rnd_time_before_reseeding;
143 integer i_seed;
144 initial begin
145     set_init_reseed_done = 0;
146     #(wait_delay);
147     #(init_delay);
148
149     #T;
150     #T;
151     #T;
152     // Initial reseed
153     for(i_seed=0;i_seed<4;i_seed=i_seed+1) begin
154         dut_seed[32*i_seed +: 32] = seed_random;
155     end
156     // $display("Reseed of PRNG requested...");
157     dut_seed_valid=1;
158     // wait for ready seed assertion
159     while(dut_seed_ready!=1) begin
160         #T;
161     end
162     #T;

```

```

163     dut_seed_valid=0;
164     // $display("New seed of PRNG fetched!");
165     set_init_reseed_done = 1;
166     #T;
167
168     // Generate random delay before next reseed
169     if (CONTINUOUS!=1) begin
170         while(1) begin
171             // wait the next execution start
172             while(dut_in_valid!=1 | dut_in_ready!=1) begin
173                 #T;
174             end
175             // Generate random latency before next reseeding
176             rnd_time_before_reseeding = {$random} % RND_RANGE_LAT_RESEED;
177             // Set new values
178             for(i_seed=0;i_seed<4;i_seed=i_seed+1) begin
179                 dut_seed[32*i_seed +: 32] = $random(seed_random);
180             end
181             for(i_seed=0;i_seed<rnd_time_before_reseeding;i_seed=i_seed+1) begin
182                 #T;
183             end
184             // Start a new reseed procedure
185             // $display("Reseed of PRNG requested...");
186             dut_seed_valid = 1;
187             // Wait for new seed eating
188             while(dut_seed_ready!=1) begin
189                 #T;
190             end
191             #T;
192             dut_seed_valid=0;
193             // $display("New seed of PRNG fetched!");
194         end
195     end
196 end
197
198 integer cycle_count;
199 always @(posedge clk) begin
200     if (syn_rst) cycle_count = 0;
201     else cycle_count = cycle_count +1;
202 end
203
204 // Generation of random ready_out (simulation of back pressure)
205 integer seed_rdy_out = 1;
206 reg [31:0] random_bit;
207 always@(posedge clk)
208 begin
209     if (CONTINUOUS==1) begin

```

```

210         random_bit <= 1;
211     end else begin
212         random_bit <= $random(seed_rdy_out);
213     end
214 end
215 `ifdef FULLVERIF
216 assign dut_out_ready = 1;
217 `else
218 assign dut_out_ready = random_bit[0];
219 `endif
220
221 always @(dut_shares_plaintext) begin
222     $display("dut_shares_plaintext=␣%x", dut_shares_plaintext);
223 end
224
225
226 // Output feeding
227 integer trash;
228 integer cnt_out;
229 integer id_out;
230 integer i;
231 integer cnt_in;
232 integer id_in;
233 integer rnd_time_before_valid_in;
234 initial begin
235     clk = 1;
236     syn_rst = 0;
237
238     dut_in_valid = 0;
239     dut_seed_valid = 0;
240
241     read_plaintext = 128'h4d29fa7f31ce1d56e3d6e11c888c6974;
242     read_umsk_key = 128'h3C4FCF098815F7ABA6D2AE2816157E2B;
243     seed_random = 0;
244 //     loadConfig();
245
246 // Init delay
247 #(wait_delay);
248 #(init_delay);
249
250 // Reset sequence
251 syn_rst = 1;
252 #T;
253 syn_rst = 0;
254 #T;
255
256 #(init_delay);

```

```

257
258 // Wait that init reseed is done
259 while(reg_init_reseed_done!=1) begin
260     #T;
261 end
262
263
264 if (CONTINUOUS==1) begin
265     rnd_time_before_valid_in = 0;
266 end else begin
267     rnd_time_before_valid_in = {$random} % RND_RANGE_LAT_IN;
268 end
269 for(i=0;i<rnd_time_before_valid_in;i=i+1) begin
270     #T;
271 end
272 // Assert valid in
273 dut_in_valid = 1;
274 while(dut_in_ready!=1) begin
275     #T;
276 end
277 #T;
278 dut_in_valid = 0;
279
280 while((dut_out_valid!=1) | (dut_out_ready!=1)) begin
281     #T;
282 end
283
284 $display("ciphertext_=%x", rec_dut_ciphertext);
285 //     saveSeed();
286 $finish;
287 end
288
289
290 endmodule

```